

1. искам да довършим архитектурата

2. ChatGPT каза:

Да. Прегледах v4.txt от репозиторието и вече виждам много ясно докъде сме стигнали. Последното решение е **още да не приемаме всички таблици за окончателни**, а да ги прегледаме

една по една.  GitHub+1

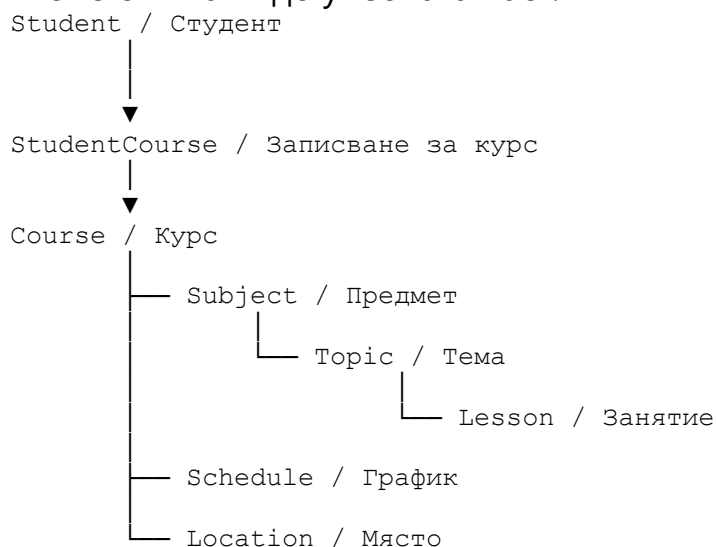
Според мен точно сега е правилният момент да **довършим архитектурата**, преди да пишем код.

Къде сме в момента

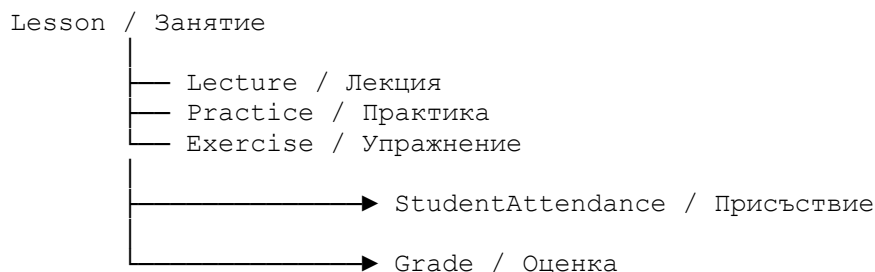
Вече имаме приети:

- Student / Студент
- Education / Образование
- Institution / Учебно заведение
- StudentEducation / Образование на студента

И сме стигнали до учебната част:



А около Lesson вече имаме:



Това е описано в последната част на v4.txt.  GitHub+1

Но виждам няколко неща, които трябва да оправим

Преди да кажем „архитектурата е готова“, аз бих спрял и бих изчистил няколко важни концепции.

1. Course ≠ CourseInstance

Това според мен е **най-важното нещо**, което липсва в сегашния модел.

Например:

C# Advanced / C# Advanced

може да е самият тип/програма на курса.

Но реално можем да го проведем:

- 01.10.2026 – 15.12.2026
- Online
- 20 студенти

и след това:

- 01.03.2027 – 15.05.2027
- Sofia
- 15 студенти

Това са **две различни провеждания на един и същ курс**.

Затова бих разгледал:

Course / Курс

↓ 1 : N

CourseInstance / Провеждане на курс

Това ще ни реши много бъдещи проблеми със:

- дата;
- място;
- преподаватели;
- график;
- записани студенти;
- капацитет;
- присъствие;
- оценки.

2. StudentCourse вероятно трябва да сочи към CourseInstance

Вместо:

Student

↓
StudentCourse

↓
Course

по-правилно би било:

Student

↓ 1:N

StudentCourse

↓ N:1

CourseInstance

↓ N:1

Course

Така Иван може да е записан в:

C# Advanced

Batch #1 / Група 1

а Мария в:

C# Advanced

Batch #2 / Група 2

без да правим два различни Course.

3. Subject / Предмет

В момента имаме:

Subject

Id

CourseId

Name

Description

Hours

IsActive

Но ако въведем CourseInstance, трябва да решим:

Subject принадлежи на Course или на CourseInstance?

Моето предложение:

Course

└─ Subject

защото учебната програма на C# Advanced е част от самия курс.

После конкретното провеждане използва тази програма.

Това ще ни позволи да имаме:

Course

C# Advanced

├─ C# OOP

├─ LINQ

├─ EF Core

└─ ASP.NET Core

и:

```
CourseInstance #1  
CourseInstance #2  
CourseInstance #3
```

да използват една и съща структура.

4. Topic / Тема

Тук сме на много добра посока:

```
Subject  
└── Topic
```

С:

```
Topic  
-----  
Id  
SubjectId  
Name  
Description  
Order  
IsActive
```

Това вече е достатъчно добро.

5. Lesson / Занятие

Тук обаче бих направил още една промяна.

В момента имаме:

```
Lesson  
-----  
Id  
SubjectId  
TopicId  
LessonType  
Date  
StartTime  
EndTime  
Hours
```

Но конкретното занятие се случва в **конкретно провеждане на курса**.

Затова:

```
CourseInstance  
    │  
    ▼  
Lesson
```

а самото занятие може да използва:

```
Subject  
Topic  
LessonType  
Date  
StartTime  
EndTime
```

Така архитектурата става логична.

Има още един важен въпрос: Grade

В v4.txt сме стигнали до:

```
Grade
-----
Id
StudentId
SubjectId
GradeValue
GradeType
Date
```



GitHub

Но аз бих го променил.

Защото оценката трябва да е свързана с **конкретното участие на студента в конкретно провеждане на курса**.

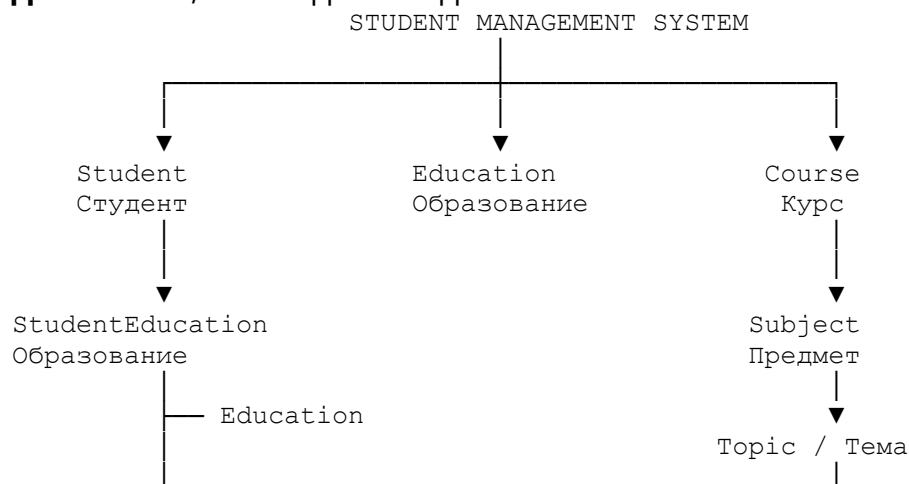
Например:

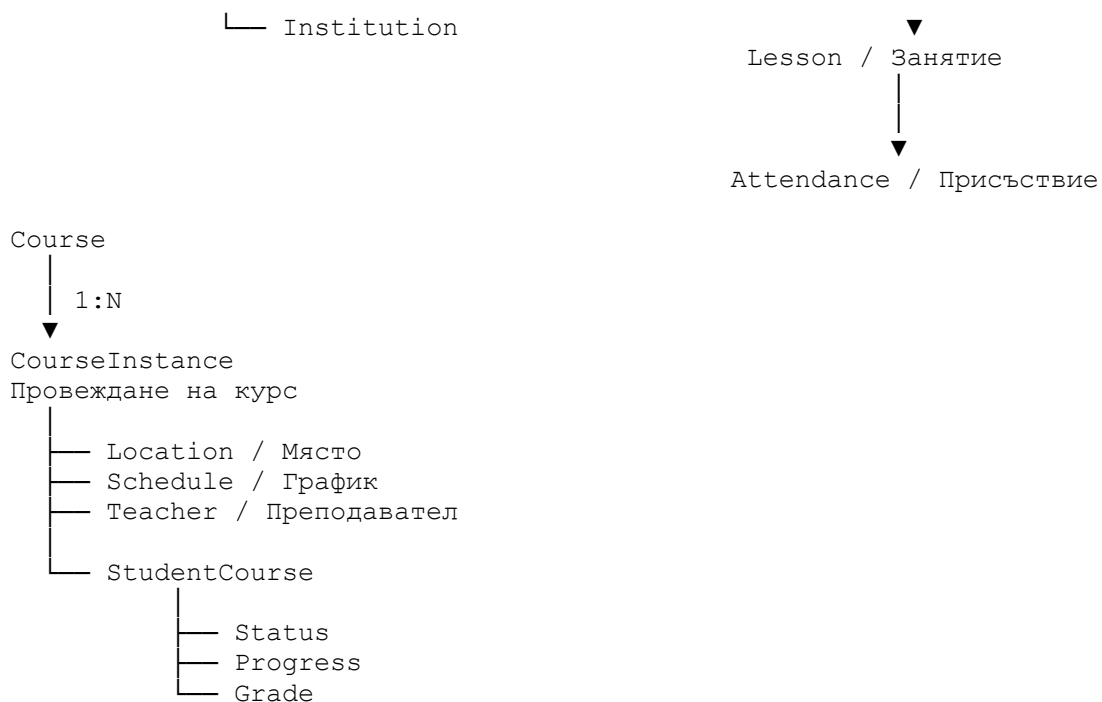
```
Student
Ivan
  |
  v
StudentCourse
  |
  v
CourseInstance #5
  |
  v
Subject
C# OOP
  |
  v
Grade
5.50
```

Това е много по-сигурно.

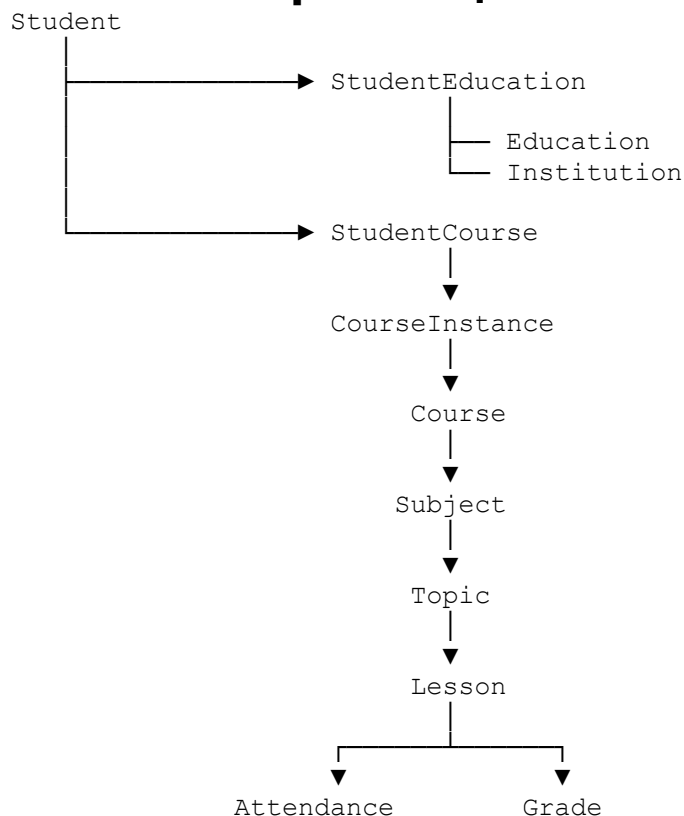
Предлагам следната архитектура

Не я приемаме още. Това е **Version 1 архитектурно предложение**, което да обсъдим:





А пълната връзка ще стане



Това вече според мен започва да изглежда като **истинска система**, а не просто CRUD приложение.

И бихме довършили архитектурата в този ред

- **Student** ☒
- **Education** ☒

- **Institution** ✓
- **StudentEducation** ✓
- **Course** ← тук сме
- **CourseInstance** ← ново, което предлагам
- **Subject**
- **Topic**
- **Lesson**
- **StudentCourse**
- **Attendance**
- **Grade**
- **Teacher**
- **Schedule**
- **Location**
- След това **общата ER схема**
- След това **Application architecture**

3. Вие казахте:

това е файл V4.txt

4. ChatGPT каза:

Да. Прати/качи съдържанието на **V4.txt** тук и ще го разгледаме като текущата версия на архитектурата.

Ще се концентрирам върху:

- каква е сегашната архитектура;
- кои части са вече добре разделени;
- къде има архитектурни проблеми или дублиране;
- какво липсва за завършена архитектура;
- как да структурираме следващата версия, **без да чупим вече направеното.**

Ако искаш, можем да го довършим стъпка по стъпка и накрая да получим ясна финална схема: **модули** → **отговорности** → **комуникация между тях** → **data flow** → **error handling** → **state management**.

5. Вие казахте:

да го направим на няколко етапа да определим нещата

6. ChatGPT каза:

Да — това е най-добрият подход. Нека **не бързаме с код**, а да заключим архитектурата на етапи. И на всеки етап записваме какво сме решили, за да не се връщаме постоянно назад.

Предлагам 6 етапа

Етап 1 — Основни понятия и граници

Първо определяме **какви са основните обекти в системата** и какво означават.

Например:

- Студент
- Образование
- Учебно заведение
- Курс
- Провеждане на курс
- Предмет
- Тема
- Занятие
- Преподавател
- Група
- Присъствие
- Оценка

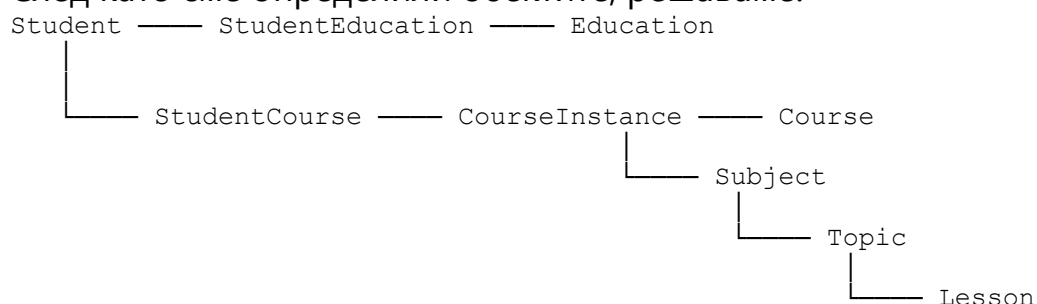
Тук **не мислим за таблици и C# класове**.

Целта е да кажем:

„Какво представлява това нещо в реалния свят?“

Етап 2 — Връзки между обектите

След като сме определили обектите, решаваме:



Тук ще определим:

- 1:1
- 1:N
- N:M
- кой притежава кой обект;
- кое е независимо;
- кое съществува само ако има родител.

Това според мен е **най-важният етап**.

Етап 3 — Жизнен цикъл

Тук питаме:

Как се появява и как изчезва всеки обект?

Например студент:

```
Student
  ↓
Active
  ↓
Inactive
  ↓
Archived
```

Курс:

```
Course
  ↓
Planned
  ↓
Active
  ↓
Completed
  ↓
Archived
```

Провеждане:

```
CourseInstance
  ↓
Scheduled
  ↓
Started
  ↓
Completed
  ↓
Cancelled
```

Тук ще решим и **къде имаме Status, къде имаме IsActive и къде изобщо не ни трябва статус**.

Етап 4 — Данни и правила

След това вече определяме полетата.

Например:

```
Student
-----
Id
FirstName
LastName
...
```

Но освен полетата ще определим и правилата:

Може ли студентът да бъде записан два пъти в едно провеждане?

Може ли да има оценка без записване в курса?

Може ли да има присъствие без Lesson?

Може ли Lesson да съществува без Topic?

Това е мястото, където архитектурата става **реална бизнес логика**, а не просто таблици.

Етап 5 — Архитектура на приложението

Едва след като домейнът е ясен, решаваме:

```
Presentation
  ↓
Application
  ↓
Domain
  ↑
Infrastructure
```

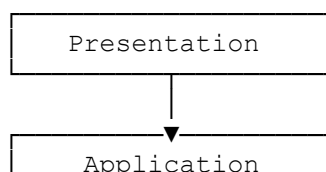
И тогава ще определим:

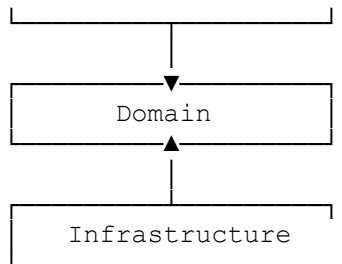
- Services
- Repositories
- DTO
- Commands / Queries
- Validation
- Exceptions
- Authentication / Authorization
- Logging
- Configuration

Тук вече ще мислим за C#/.NET архитектурата.

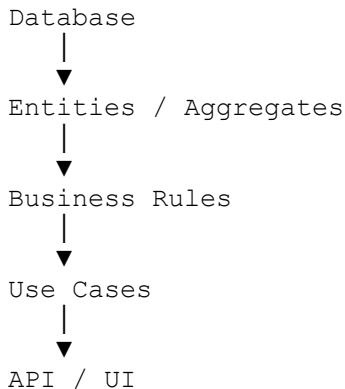
Етап 6 — Финална архитектура

Накрая събираме всичко:





и ОТДЕЛНО:



Тогава ще можем да кажем:

Архитектурата е заключена.

И чак след това започваме имплементацията.

Важното

Предлагам на всеки етап да работим по следния начин:

- 1. Предлагам вариант.**
- 2. Обсъждаме го.**
- 3. Ти го одобряваш или променяме.**
- 4. Маркираме решението като APPROVED.**
- 5. Не го променяме на следващите етапи, освен ако не открием сериозен архитектурен проблем.**

Така няма да се получи „днес решихме едно, след 20 съобщения го забравихме и построихме друго“.

Започваме с Етап 1

Нека първо определим само основните понятия в Student Manager.

Аз ще взема **V4.txt** като отправна точка, а не ще измислям системата от нулата.

Първият въпрос е:

Кои са според теб основните обекти, които задължително трябва да съществуват в системата?

Можеш просто да ги изредиш, дори и да не си сигурен за някои.

После ще ги разгледаме **един по един**.

1. да ги караме точка по точка важни концепции и така да изчистим архитектурата

2. ChatGPT каза:

Точно така. 👍 Това е по-добрият начин.

Ще вървим **точка по точка**, като всяка точка е отделна архитектурна концепция. Няма да прескачаме към следващата, докато не изчистим текущата.

Точка 1 — Course и CourseInstance

Това е първата концепция, която според мен трябва да изясним, защото от нея

ЗАВИСЯТ StudentCourse, Lesson, Schedule, Teacher, Attendance и Grade.

Въпросът

Имаме:

Course — Курс

Това е самият курс като учебно съдържание/предложение.

Например:

C# Advanced

След това имаме конкретно провеждане:

C# Advanced

Група 1

01.10.2026 → 15.12.2026

вечерна група

20 студенти

и:

C# Advanced

Група 2

01.03.2027 → 15.05.2027

дневна група

15 студенти

И двете са един и същ **Course**, но са две различни **CourseInstance**.

Моето предложение

Course

↓
1 : N

CourseInstance

Course съдържа **какво се изучава**.

CourseInstance съдържа **конкретно кога, къде и с кого се провежда**.

Например:

Course

Id
Name
Description
Code

IsActive

a:

CourseInstance

Id
CourseId
StartDate
EndDate
Capacity
Status
...

След това:

Student



StudentCourse



CourseInstance

а не:

Student



StudentCourse



Course

Това ни дава много по-чист модел.

Но има един важен въпрос

Трябва да решим **какво точно представлява Course в нашата система.**

Има поне два варианта:

Вариант А — Course = учебен продукт

Course

"C# Advanced"



CourseInstance #1

CourseInstance #2

CourseInstance #3

Вариант В — Course = конкретен курс

Тогава изобщо няма CourseInstance и всеки курс е конкретно провеждане.

Аз лично съм **за Вариант А**, но не искам да го приемаме автоматично.

Затога засега само един въпрос:

В нашия Student Manager искаме ли един и същ курс да може да бъде провеждан многократно — с различни групи, периоди, преподаватели, места и студенти?

Ако отговорът е **да**, приемаме концепцията Course → CourseInstance И продължаваме към следващия въпрос.

1. имената съдържат английски/ български име за полесно да се ориентирам къде сме

2. **ChatGPT каза:**

Да, разбрах. 👍 И това е добра идея.

Оттук нататък при всяка архитектурна точка ще изписвам **English / Български**, за да е ясно за кой елемент говорим.

Например:

Точка 1 — Course / Курс и CourseInstance / Провеждане на курс

Ще използвам имената така:

- **Course / Курс**
- **CourseInstance / Провеждане на курс**
- **Student / Студент**
- **StudentCourse / Записване на студент в курс**
- **Subject / Предмет**
- **Topic / Тема**
- **Lesson / Занятие**
- **Teacher / Преподавател**
- **Attendance / Присъствие**
- **Grade / Оценка**

И когато обсъждаме конкретно поле:

- **Courseld / Идентификатор на курса**
- **StartDate / Начална дата**
- **EndDate / Крайна дата**
- **Status / Статус**

Ще запазим **English името като техническо име**, а българското ще е за ориентация и обсъждане.

И още нещо — когато приключим дадена точка, ще я маркираме например:

 **APPROVED — Course / Курс → CourseInstance / Провеждане на курс**

Така постепенно ще изградим един **Architecture Decision Log**, без да губим вече взетите решения.

Продължаваме с **Точка 1 — Course / Курс и CourseInstance / Провеждане на курс** и първо решаваме какво означава Course / Курс в нашата система.

3. Вие казахте:

може ли да го направим така

4. ChatGPT каза:

Да, разбира се. 👍 Само ми покажи **точно как си го представяш** — може да е пример, схема или просто няколко реда текст.

Ще го приема като **формата, по която ще работим оттук нататък**, и ще се придържам към нея за всяка следваща архитектурна точка.

5. Вие казахте:

с v1 - ще се отнася за номер и версия на проекта

6. ChatGPT каза:

Да, напълно. Това даже ще ни помогне много да следим развитието на архитектурата.

Можем да използваме формат:

V1 — Версия 1 на проекта

А вътре във V1 да имаме отделни точки:

```
v1
├── 1. Course / Курс
├── 2. CourseInstance / Провеждане на курс
├── 3. Student / Студент
├── 4. StudentCourse / Записване на студент в курс
└── ...
```

Но бих предложил още по-добре да разделим **версията на проекта** от **архитектурните точки**:

```
v1
├── V1.1 — Course / Курс
├── V1.2 — CourseInstance / Провеждане на курс
├── V1.3 — Student / Студент
├── V1.4 — StudentCourse / Записване на студент в курс
└── ...
```

Така например, ако обсъждаме:

V1.2 — CourseInstance / Провеждане на курс

ще знаем едновременно:

- v1 → версията на проекта;
- 2 → втората архитектурна точка;

- CourseInstance / Провеждане на курс → какво точно разглеждаме.

А когато приключим цялата архитектура на V1, можем да кажем:

V1 — Architecture APPROVED

След това при сериозна промяна:

V2 — Architecture

и започваме нова версия, без да губим V1.

Харесва ми този вариант и бих го използвал оттук нататък.

7. Вие казахте:

в v1 - ще е по общо например:

8. ChatGPT каза:

Да, точно така. Разбирам те — **V1 да бъде общата версия/етап**, а не всяка отделна точка да получава номер като V1.1.

Покажи ми примера след „например:“ и ще се съобразя **точно с твоята схема**.

9. Вие казахте:

курса ще съдържа 2 или 3 таблици

10. ChatGPT каза:

Да — вече те разбирам. 👍

Тоест **V1** е общата концепция, а вътре в нея разглеждаме отделните части.

Например:

V1 — Course / Курс

Course / Курс като концепция може да бъде реализиран с **2 или 3 таблици**.

V1 — Course / Курс

```
|
├─ Course / Курс
├─ ...
└─ ...
```

Идеята е **първо да определим концепцията**, а след това да решим колко таблици са необходими за нея.

Това е по-добре от това предварително да кажем:

„Course = една таблица“

защото една бизнес концепция може напълно нормално да се нуждае от 2–3 таблици.

Предлагам да работим така

За всяка V#:

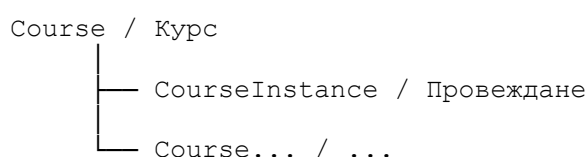
- **Каква е концепцията?**

- Какво трябва да може да прави?
- Кои са необходимите таблици?
- Каква е ролята на всяка таблица?
- Как са свързани таблиците?
- Какви са основните правила и ограничения?
- Накрая: **APPROVED / ОДОБРЕНО**

И чак тогава преминаваме към следващата V#.

Така например:

V1 — Course / Курс



Не приемам още конкретните 2–3 таблици — първо ги обсъждаме и решаваме защо са нужни.

Кажете ми сега как си представяш **V1 — Course / Курс** и кои 2–3 таблици имаш предвид.

11. Вие казахте:

1/ име на модул - описание и критерии да се запише

12. ChatGPT каза:

Да. Сега вече схемата е ясна. 👍

Искаш **V1 да представлява модул/концепция**, а вътре да запишем **какво представлява модулът, какво трябва да постигне и по какви критерии ще приемем архитектурата му**.

Например:

V1 — Course / Курс

1. Име на модула

Course / Курс

2. Описание

Тук описваме с няколко изречения какво представлява модулът и за какво служи.

3. Критерии

Тук записваме конкретните условия, които трябва да бъдат изпълнени, за да кажем, че модулът е архитектурно завършен.

Например:

V1 — Course / Курс

Име на модул:

Course / Курс

Описание:

[какво представлява модульът]

Критерии:

1. ...
2. ...
3. ...
4. ...

Статус:

IN PROGRESS

След обсъждането:

Статус:

APPROVED

И едва след това преминаваме към V2.

Така ще можем постепенно да изградим цялата архитектура:

V1 – Course / Курс

V2 – Student / Студент

V3 – Education / Образование

V4 – Institution / Учебно заведение

...

И всяка версия ще има собствено **описание + критерии за приемане**.

Това според мен е много по-чист подход.

Готов съм — започваме с V1. Кажете ми какво искате да запишем като **име на модула**, а после ще формулираме заедно описанието и критериите.

13. Вие казахте:

под модул/таблица/ е например: c#, js; python кратко описание, какви са изискванията да се запише този модул/английски или нещо друго

14. ChatGPT каза:

Да, сега те разбрах напълно. 👍

Тук „**модул**“ не означава функционален модул на системата, а **учебен модул/тема, която студентът изучава**.

Например:

V1 – Course / Курс

Модул / Module:

C#

Кратко описание:

Основи на програмирането с C#.

Изисквания за записване:

- ...
- ...

Друг запис:
V2 — Course / Курс

Модул / Module:
JavaScript

Кратко описание:
Основи на JavaScript и работа с ...

Изисквания за записване:
- ...
- ...

И:
V3 — Course / Курс

Модул / Module:
Python

Кратко описание:
Основи на програмирането с Python.

Изисквания за записване:
- ...
- ...

Тоест **Module / Модул** ще има примерно:

Поле	Предназначение
Name / Име	C#, JavaScript, Python и т.н.
Description / Описание	Кратко описание на модула
Requirements / Изисквания	Какво трябва да знае/има студентът
Language / Език	C#, JavaScript, Python... или друг тип
IsActive / Активен	Дали модулят може да бъде използван

Но тук има **много важен архитектурен въпрос**:

Дали C#, JavaScript, Python са **самите Module / Модули**, или са **Language / Езици**, а модулят е нещо като:

C# Fundamentals / Основи на C#
C# Advanced / Напреднал C#
JavaScript Fundamentals / Основи на JavaScript
Python Fundamentals / Основи на Python

Това трябва да решим **преди да определяме таблиците**.

Аз бих започнал точно от това:

V1 — Module / Модул

Въпрос 1:

C#, JavaScript, Python — това ли са **самите Module / Модули**, или искаме отделно понятие **ProgrammingLanguage / Програмен език**?

Това решение ще ни помогне много за останалата архитектура.

15. Вие казахте:

правим да не е насочена точно към определена област, а да може да се използва от голяма аудитория

16.ChatGPT каза:

Да — това е **много важно архитектурно решение**. Не трябва да изграждаме Student Manager като система само за програмиране. Тогава C#, JavaScript, Python са **само примери за съдържание**, а не част от самата архитектура.

V1 — Module / Модул

Име на модула

Module / Модул

Кратко описание

Module / Модул представлява самостоятелна учебна единица, която може да бъде използвана в различни области на обучение.

Модулът **не трябва да бъде обвързан с конкретна професия, предмет или технологична област**.

Примери:

- C# / C#
- Python / Python
- English Grammar / Английска граматика
- Mathematics / Математика
- Accounting / Счетоводство
- Digital Marketing / Дигитален маркетинг
- Photography / Фотография

Тоест системата трябва да позволява:

Module / Модул

```
— Programming / Програмиране
— Languages / Езици
— Mathematics / Математика
— Business / Бизнес
— Arts / Изкуства
— ...
```

без архитектурата да знае предварително какви ще бъдат тези области.

Основен принцип

Student Manager управлява обучението, а не конкретна област на знанието.

Това означава, че няма да правим неща като:

```
CSharpModule
JavaScriptModule
PythonModule
```

или:

```
ProgrammingCourse
EnglishCourse
```

В основния модел.

Вместо това ще имаме **общ модел**, който позволява всички тези случаи.

Критерии за V1

Модулът трябва:

- Да има **име**.
- Да има **кратко описание**.
- Да може да има **изисквания за записване**.
- Да не бъде обвързан с конкретна професия или област.
- Да може да се използва от различни видове обучения.
- Да може да бъде използван от голяма и различна аудитория.
- Да позволява бъдещо разширяване без промяна на основната концепция.
- Да поддържа съдържание на различни езици, ако решим това да е необходимо.

И тук има едно важно нещо, което **още не трябва да решаваме**: Дали Module / Модул ще бъде една таблица или ще има 2–3 свързани таблици.

Това ще го решим **след като уточним самата концепция**.

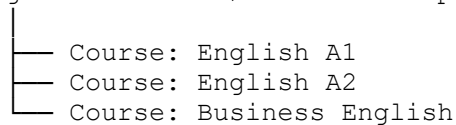
Следващият въпрос за V1

Искам да изясним само **едно нещо**:

Може ли един Module / Модул **да бъде част от повече от един** Course / Курс?

Например:

Module: English Grammar / Английска граматика



Ако да — това ще има много важно значение за връзката **Module / Модул ↔ Course / Курс**.

17. Вие казахте:

може да го направим както е в реалността: да направим свързваща таблица. от нея да се избират кои курсове влизат и кой не влизат в този модул

18. ChatGPT каза:

Да. **Това е правилният подход**, ако искаме моделът да е универсален и да отговаря на реалната организация на обученията.

Тогава не правим `Module → Course` като директна зависимост, а **свързваща таблица**.

V1 — Module / Модул

Концепция

Имаме:

`Module / Модул`



`ModuleCourse / Модул-Курс`



`Course / Курс`

Тоест:

`Module / Модул`

`ModuleCourse — Course / Курс`

`ModuleCourse — Course / Курс`

`ModuleCourse — Course / Курс`

Това е **N:M връзка**.

Пример

Имаме модул:

`Module`

Name: `Programming / Програмиране`

Description: ...

Requirements: ...

Към него можем да включим:

`ModuleCourse`

ModuleId CourseId

1 10

1 11

1 15

което означава:

`Programming / Програмиране`

`C# Fundamentals`

`Python Fundamentals`

`JavaScript Fundamentals`

Но друг курс:

`Java Fundamentals`

може да **не бъде включен** в този модул.

И това е хубавото — **не променяме** Course / Курс, за да го включим или изключим от даден модул.

Таблиците за V1

Засега виждаме минимум:

1. Module / Модул

Съдържа информацията за самия модул.

Примерно:

```
Module
-----
Id
Name
Description
Requirements
...
```

2. Course / Курс

Това е самостоятелната концепция за курс.

```
Course
-----
Id
Name
Description
...
```

3. ModuleCourse / Модул-Курс

Свързва двете:

```
ModuleCourse
-----
Id
ModuleId
CourseId
...
```

Тук вече можем да имаме и допълнителна информация за връзката, ако се окаже необходима.

Например:

```
IsRequired / Задължителен
Order / Ред
StartDate / Начална дата
EndDate / Крайна дата
```

Но **нека не ги добавяме автоматично**. Първо трябва да решим дали реално принадлежат на тази връзка.

Важното архитектурно решение

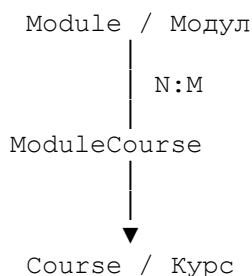
С това решение казваме:

Module / Модул не притежава Course / Курс.

Те са независими концепции.

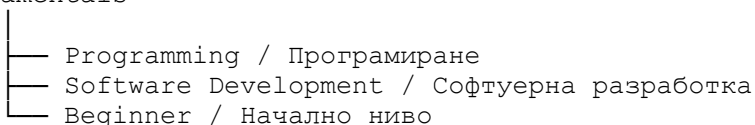
ModuleCourse / Модул-Курс определя **кои курсове участват в конкретния модул.**

Това ни дава:



И позволява един курс да участва в няколко модула:

C# Fundamentals



Това вече е доста универсален модел.

V1 — текущо решение

Module / Модул

→ самостоятелна учебна концепция.

Course / Курс

→ самостоятелна учебна концепция.

ModuleCourse / Модул-Курс

→ свързваща таблица, която определя участието на даден курс в даден модул.

Статус: IN PROGRESS

Следващото, което бих изчистил, е **какво точно съдържа** Module / Модул — име, описание, изисквания и т.н. Да ги определим **поле по поле**, преди да решаваме окончателно таблицата.

19. Вие казахте:

във в1 ще са само курсовете, а във в2 ще включим и модулите

20. ChatGPT каза:

Да — така вече е много по-ясно. И според мен е по-добре да го разделим именно така.

V1 — Course / Курс

В V1 разглеждаме **само курсовете**.

Не включваме още:

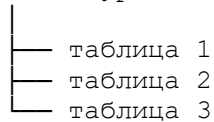
- Module / Модул
- ModuleCourse / Модул-Курс
- други концепции, свързани с групиране на курсове.

Целта на V1 е да изчистим **какво представлява един Course / Курс** и какви таблици са необходими за него.

Например:

V1 — Course / Курс

Course / Курс



Тук ще определим:

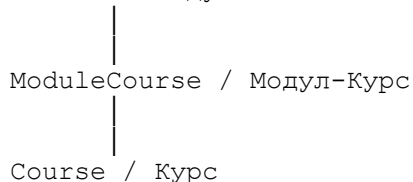
- какво е курс;
- какви данни има;
- кои таблици са необходими;
- как са свързани;
- какви са изискванията;
- какви са правилата;
- какво означава курсът в реалността.

След като **V1 бъде APPROVED**, преминаваме към:

V2 — Module / Модул

Тогава вече въвеждаме:

Module / Модул



и решаваме как модулът избира кои курсове участват в него.

Така **V2 използва вече готовия V1**, вместо да смесваме двете концепции от самото начало.

Значи засега фиксираме:

V1 = Course / Курс

V2 = Module / Модул + връзката му с Course / Курс

Това е първото архитектурно решение, което можем да считаме за прието.

Следва V1 — Course / Курс: да определим какво точно представлява курсът и кои 2–3 таблици реално са му нужни.

21. Вие казахте:

дай пример първо кви полета/редове да съдържа

22.ChatGPT каза:

Да. Нека за **V1 — Course / Курс** първо направим само примерен модел, без още да го приемаме за окончателен.

Аз бих започнал с **една основна таблица** Course / Курс, а после ще видим дали са нужни още 1–2 таблици.

V1 — Course / Курс

Course / Курс

Поле / Field	Пример	Предназначение
Id	1	Уникален идентификатор
Code	CS-BASIC-01	Кратък код на курса
Name	C# Основи	Име на курса
Description	Въведение в програмирането с C#	Кратко описание
Requirements	Основни компютърни умения	Изисквания за записване
Level	Beginner	Ниво на курса
Duration	120	Продължителност
DurationUnit	Hours	Мерна единица — часове/дни и т.н.
Language	bg	Език на курса
Capacity	20	Максимален брой участници
IsActive	true	Дали курсът е активен
CreatedAt	2026-09-04	Създаване
UpdatedAt	2026-09-04	Последна промяна

Но тук има няколко неща, които трябва да обсъдим

Например Capacity / Капацитет **може би не трябва да е в Course / Курс.**

Ако имаме:

Course / Курс

C# Основи

и го провеждаме:

2026 → 20 студенти

2027 → 30 студенти

2028 → 15 студенти

тогава Capacity всъщност принадлежи на **конкретното провеждане**, а не на самия курс.

Същото може да се окаже вярно за:

- StartDate / Начална дата
- EndDate / Крайна дата
- Teacher / Преподавател
- Location / Място
- Schedule / График
- Capacity / Капацитет

Затова **нека не ги слагаме автоматично в Course.**

По-чист първоначален вариант

Course / Курс

Id
Code
Name
Description
Requirements
Level
Duration
DurationUnit
Language
IsActive
CreatedAt
UpdatedAt

А конкретното провеждане евентуално ще бъде отделна концепция.

И още нещо важно: Requirements / Изисквания **може да се окаже отделна таблица**, ако искаме повече от един критерий.

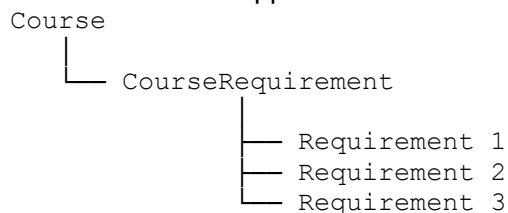
Например:

C# Основи

Изисквания:

- Основни компютърни умения
- Познания по математика
- Английски език A2

Тогава можем да имаме:



И точно такива решения искам да изчистим **преди да кажем, че V1 е готов.**

Първото нещо, което бих решил сега, е Course / Курс да съдържа ли само информацията за самия курс, или искаме част от тези полета да бъдат в отделна таблица.

23. Вие казахте:

тъй като той може да е 3: 1/онлайн 2/присъствен и 3/смесен да им дадем да може да те да си определят капацитета

24. ChatGPT каза:

Да — точно тук вече навлизаме в реалното поведение на **Course / Курс**.

Ако един курс може да се провежда по различен начин:

- **Online / Онлайн**

- **On-site / Присъствен**
- **Hybrid / Смесен**

тогава според мен **Capacity / Капацитет** трябва да може да се определя според конкретния начин на провеждане.

Например:

Course / Курс

C# Основи

DeliveryMode / Начин на провеждане:

- Online / Онлайн
- On-site / Присъствен
- Hybrid / Смесен

И капацитетът може да е различен:

Online / Онлайн → 100

On-site / Присъствен → 20

Hybrid / Смесен → 50

Но тук има една важна архитектурна подробност.

Не бих направил:

OnlineCapacity

OnsiteCapacity

HybridCapacity

директно в Course / Курс, защото това ще ни заключи модела към точно тези три начина.

По-универсално е:

Course / Курс



CourseDelivery / Начин на провеждане

Пример:

CourseDelivery	Capacity
Online / Онлайн	100
On-site / Присъствен	20
Hybrid / Смесен	50

Така ако след време добавим:

- Self-paced / Самостоятелно обучение
- Workshop / Работилница
- Blended / Комбинирано

не променяме основната структура.

Но трябва да уточним нещо

Има разлика между:

Course / Курсът позволява Online

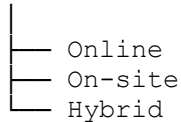
и:

конкретното провеждане на курса е Online с капацитет 100.

Например:

Course / Курс

C# Основи



но конкретно:

CourseInstance / Провеждане

C# Основи — октомври 2026

Mode: Online

Capacity: 100

Така според мен моделът ще бъде най-близо до реалността.

Тоест предлагам DeliveryMode / Начин на провеждане да е отделна концепция, а Capacity / Капацитет да принадлежи на конкретното провеждане.

Ако си съгласен с тази логика, това може да стане първото правило на **V1 — Course / Курс.**

25. Вие казахте:

ако някой потребител избере смесен бих го направила ако се избере да може да избира колко ще са онлайн и колко ще са присъствено

26. ChatGPT каза:

Да — това вече е много по-близо до реалното поведение. И според мен трябва да го заложим в архитектурата.

Ако потребителят избере:

Hybrid / Смесен

системата не трябва просто да записва Hybrid = true, а да му даде възможност да определи капацитета по начин на участие.

Пример

Потребителят създава курс:

Course / Курс

C# Основи

Избира:

DeliveryMode / Начин на провеждане

☐ Online / Онлайн

☐ On-site / Присъствен

☒ Hybrid / Смесен

След избора на Hybrid / Смесен се показва:

Hybrid Capacity / Капацитет на смесения курс

Online / Онлайн: [20]

On-site / Присъствен: [10]

Общ капацитет: 30

Така:

20 Online

10 On-site

30 Total

Но бих направил още една стъпка

Не бих записвал само:

Mode = Hybrid

OnlineCapacity = 20

OnsiteCapacity = 10

защото вече виждаме, че **начинът на провеждане има собствена информация.**

По-логично:

Course / Курс



CourseDelivery / Начин на провеждане



Online / Онлайн

Capacity: 20

On-site / Присъствен

Capacity: 10

При чисто онлайн:

Course



CourseDelivery



Online

Capacity: 100

При чисто присъствен:

Course



CourseDelivery



On-site

Capacity: 25

При смесен:

Course



CourseDelivery



Online

Capacity: 20



CourseDelivery



On-site

Capacity: 10

И това ни дава хубава възможност за бъдещето

Потребителят може да промени:

Online: 20

On-site: 10

на:

Online: 30

On-site: 5

без да променяме структурата на Course / Курс.

А системата автоматично показва:

Total Capacity / Общ капацитет = 35

Само че тук се появява един важен въпрос за V1
Капацитетът е максималният брой студенти, които курсът
принципно може да има, или е капацитетът на конкретното
провеждане?

Защото ако курсът C# Основи има:

Online: 100

On-site: 20

това може да е **характеристика на курса**.

Но ако октомври 2026 е:

Online: 20

On-site: 10

а януари 2027:

Online: 30

On-site: 15

тогава капацитетът принадлежи на **CourseInstance / Провеждане**
на курс.

Аз бих избрал **второто**, защото е по-гъвкаво и отговаря по-добре
на реалността.

Това означава, че във V1 ще определим **какво може курсът**, а
по-нататък ще определим **как конкретно се провежда курсът**.

27. Вие казахте:

ако правим/водим статистика кое би предложил

28. ChatGPT каза:

Да — ако мислим **от самото начало и за статистика**, бих избрал
модел, който пази **реалните участия**, а не само крайния
капацитет.

Моето предложение

Да разделим:

- **Course / Курс** → какво представлява курсът.
- **CourseInstance / Провеждане на курс** → конкретното
провеждане.
- **CourseDelivery / Начин на участие** → Online / Онлайн, On-
site / Присъствен.
- **Enrollment / Записване** → конкретният студент в
конкретното провеждане.

Тогава статистиката става много добра.

Пример

Имаме:

Course / Курс:

English A1 / Английски A1

Конкретно провеждане:

CourseInstance / Провеждане

2026-10-01 → 2026-12-20

Капацитет:

Online / Онлайн 20

On-site / Присъствен 10

Записвания:

Online:

15 студенти

On-site:

8 студенти

Системата може да изчисли:

Общ капацитет: 30

Общо записани: 23

Свободни места: 7

Online:

Капацитет: 20

Записани: 15

Запълване: 75%

On-site:

Капацитет: 10

Записани: 8

Запълване: 80%

И след това можем да правим статистика:

- колко курса са проведени;
- колко студенти са участвали;
- Online срещу On-site;
- среден процент запълване;
- най-популярни курсове;
- отпаднали студенти;
- завършили студенти;
- средна оценка;
- присъствие;
- успеваемост;
- брой записвания по период;
- натоварване на преподаватели;
- използване на местата.

Особено важно при Hybrid / Смесен

Аз не бих записвал само Hybrid.

Бих записвал реалните компоненти:

CourseDelivery

Id

CourseInstanceId

DeliveryType

Capacity

Например:

1 | 100 | Online | 20

2 | 100 | On-site | 10

Така Hybrid / Смесен всъщност се получава от наличието на двата начина.

Това е по-силно архитектурно решение, защото статистиката не трябва да „гадае“ какво означава Hybrid.

И още нещо много важно

При записването:

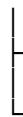
Enrollment / Записване

трябва да знаем **как студентът участва:**

Student



Enrollment



CourseInstance

CourseDelivery



Online

On-site

Тогава можем да кажем:

Иван е записан в English A1, провеждане №15, **Online**.

А Мария:

Мария е записана в същото провеждане, **On-site**.

Това е изключително полезно за статистиката.

Затова бих заложил това като архитектурен принцип

Всичко, което искаме да измерваме в бъдеще, трябва да има реално записано събитие/състояние, а не да бъде изчислявано от свободен текст или общо поле.

Например не:

Mode = Hybrid

а:

Online → Capacity 20

On-site → Capacity 10

От това системата сама може да изведе:

Mode = Hybrid

Total Capacity = 30

Това е много по-добро за **статистики, справки и бъдещи анализи.**

За V1 бих записал именно тази идея като

изискване: архитектурата трябва да позволява надеждна статистика на ниво **CourseInstance /**

Провеждане, CourseDelivery / Начин на участие и Enrollment / Записване.

А след това можем да решим **точно кои таблици влизат във V1**, без още да ги измисляме предварително.

29. Вие казахте:

мисля че вече почти го направихме като разбрахме кво точно искаме

30. ChatGPT каза:

Да, точно. 👍 Вече започнахме от **реалното поведение на системата**, а не от „какви таблици да направим“. Това е правилният ред.

В момента за **V1 — Course / Курс** имаме доста ясна концепция.

V1 — Course / Курс — досегашни решения

1. Course / Курс

Курсът е общото учебно предложение.

Пример:

C# Основи

Английски A1

Счетоводство за начинаещи

Фотография

Той **не е конкретното провеждане.**

2. CourseInstance / Провеждане на курс

Един Course / Курс може да има много конкретни провеждания.

C# Основи

- Провеждане 01 / 2026
- Провеждане 02 / 2027
- Провеждане 03 / 2027

Това позволява различни дати, преподаватели, капацитети и т.н.

3. CourseDelivery / Начин на участие

Не заключваме системата само в:

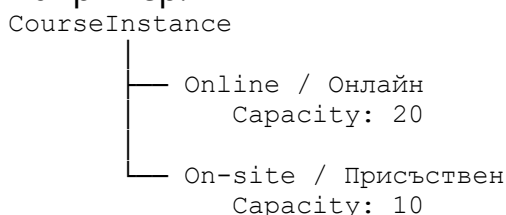
Online

On-site

Hybrid

а позволяваме конкретното провеждане да има един или повече начина.

Например:



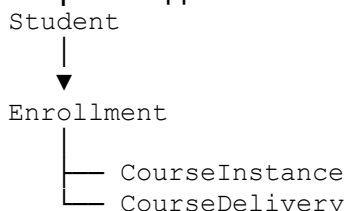
От това системата разбира, че провеждането е **Hybrid / Смесено**.

4. Enrollment / Записване

Записването трябва да е към **конкретно CourseInstance /**

Провеждане, а не просто към Course.

И трябва да знаем начина на участие:



Така статистиката ще бъде достоверна.

А най-хубавото е, че вече имаме критерий

Не питаме само:

„Имаме ли таблица за това?“

А:

„Можем ли от записаните данни да възстановим какво реално се е случило?“

Например:

На 15 октомври 2026 г. имаме курс English A1, провеждане №12.

20 места Online, 10 места On-site.

17 студенти са записани Online, 8 On-site.

2 от Online са отпаднали.

Средната оценка е 5.42.

Ако архитектурата може надеждно да даде тази информация — значи сме я направили добре.

Предлагам следващата стъпка

Сега **да не добавяме повече концепции**.

Да направим едно последно нещо за V1:

да превърнем всичко, което току-що решихме, в окончателен списък от таблици и полета.

Ще минем таблица по таблица:

V1 — Course / Курс

1. Course / Курс
2. CourseInstance / Провеждане на курс
3. CourseDelivery / Начин на участие
4. Enrollment / Записване

и за всяка ще определим:

- поле;
- тип;
- задължително/незадължително;
- връзки;
- правила.

След това **V1 = APPROVED** и чак тогава започваме **V2 — Module / Модул**.

Така няма да се разпилеем и ще запазим архитектурата чиста.

31. Вие казахте:

за всеки студент трябва да се пази информация за курса кво е записал, кво е завършил, кое учи, колко му остава/ в точки: (7 от 10) и проценти/, междинна оценка и крайна оценка

32. ChatGPT каза:

Да. Това вече е **много важна част от V1**, защото ни показва какво трябва да пазим за връзката между **Student / Студент** и **CourseInstance / Провеждане на курс**.

И тук бих направил едно важно разграничение: **не всичко трябва да се съхранява като отделно поле**. Част от информацията може да се изчислява.

V1 — StudentCourse / Обучение на студент в курс

За всеки студент трябва да можем да отговорим на:

- Какъв курс е записал?
- Кое конкретно провеждане?
- Кога се е записал?
- Започнал ли е?

- Учи ли го в момента?
- Завършил ли го е?
- Колко е завършил?
- Колко му остава?
- Какъв е процентът?
- Каква е междинната оценка?
- Каква е крайната оценка?

Пример

Student / Студент
Иван Иванов

Course / Курс
C# Основи

CourseInstance / Провеждане
Група 2026-10

Статусът може да изглежда:

Status / Статус:
InProgress / В процес

Progress / Напредък:
7 / 10

Percentage / Процент:
70%

Remaining / Остава:
3 / 10

IntermediateGrade / Междинна оценка:
5.25

FinalGrade / Крайна оценка:
—

След завършване:

Status:
Completed / Завършен

Progress:
10 / 10

Percentage:
100%

Remaining:
0 / 10

IntermediateGrade:
5.25

FinalGrade:
5.75

Но има една много важна архитектурна подробност

Не бих пазил едновременно:

```
Progress = 7
Total = 10
Percentage = 70%
Remaining = 3
```

като четири независими стойности.

Защото може да стане:

```
Progress = 7
Total = 10
Percentage = 80%
Remaining = 2
```

и данните вече са противоречиви.

По-добре да имаме **един източник на истината**.

Например:

```
CompletedUnits = 7
TotalUnits = 10
```

и системата изчислява:

```
Percentage = 70%
Remaining = 3
```

Още по-добре

Ако по-надолу имаме реални Lesson / Занятие, тогава изобщо не е
нужно да казваме:

7 от 10

на ръка.

Можем да получим:

```
Total lessons:      10
Completed lessons:   7
Remaining lessons:   3
Progress:            70%
```

Това е много по-надеждно.

За оценките

Тук бих разделил:

IntermediateGrade / Междинна оценка

и

FinalGrade / Крайна оценка

Но трябва да определим **откъде се получават**.

Например:

```
Assessment / Оценяване
├── Midterm / Междинно
└── Final / Крайно
```

Защото в реален курс може да има:

Quiz 1

Quiz 2
Практическа работа
Midterm
Final exam

и тогава:

IntermediateGrade = 5.25

може да е изчислена стойност от няколко оценки.

Същото за крайната.

Така бих го разделил концептуално

Student / Студент



Enrollment / Записване



CourseInstance / Провеждане

Status / Статус

Progress / Напредък

IntermediateGrade / Междинна оценка

FinalGrade / Крайна оценка

А Progress / Напредък да се **изчислява от реално завършеното обучение**, вместо да го дублираме в няколко полета.

И това добавя още един важен критерий към V1

За всеки Student / Студент системата трябва по всяко време да може да покаже пълното му състояние спрямо конкретен Course / Курс — записан, текущо обучаващ се, завършил, напредък, оставащо обучение, междинна и крайна оценка.

Това вече е доста добър критерий за архитектурата.

Следващото, което бих изчистил, е **какво точно означава** 7 от

10 — дали 10 са Lesson / Занятия, Topic / Теми, Exam / Изпити, ИЛИ

някакви **CourseUnit / Учебни единици**. Това е важно, защото ще определи как изчисляваме Progress / Напредък.

33. Вие казахте:

забравих и още едно: присъствие на занятията изразени: 7 от 10 и проценти

34. ChatGPT каза:

Да, това задължително трябва да го добавим. И всъщност тук се оформя доста добре какво трябва да знаем за **Student / Студент** във връзка с конкретен курс.

V1 — StudentCourse / Обучение на студент в курс

За всеки студент и конкретно CourseInstance / Провеждане на курс трябва да можем да покажем:

1. Course status / Статус на курса

Enrolled / Записан
InProgress / Обучава се
Completed / Завършил
Cancelled / Отказан

2. Course progress / Напредък в курса

Например:

7 / 10
70%

Тоест:

Completed: 7
Total: 10
Progress: 70%
Remaining: 3

3. Attendance / Присъствие

Това е **отделно** от Progress / Напредък.

Например:

Attendance / Присъствие

Присъствал: 7 / 10
Процент: 70%
Отсъствал: 3

Това е важно, защото студент може да има:

Progress: 90%
Attendance: 70%

или:

Progress: 70%
Attendance: 100%

Двете неща измерват различни характеристики.

4. Grades / Оценки

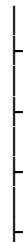
IntermediateGrade / Междинна оценка: 5.25
FinalGrade / Крайна оценка: 5.75

Получаваме следната картина

Student / Студент



Enrollment / Записване

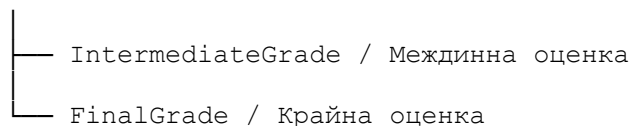


CourseInstance / Провеждане

Status / Статус

Progress / Напредък

Attendance / Присъствие



И на екрана можем да покажем например:

C# Advanced		
Status:	In Progress	
Progress:	7 / 10	(70%)
Remaining:	3	
Attendance:	8 / 10	(80%)
Midterm:	5.25	
Final:	—	

И тук има много важен принцип

Както при Progress, **не е необходимо да пазим всички изчислени стойности.**

Ако имаме реалните данни:

Lessons = 10

CompletedLessons = 7

AttendedLessons = 8

можем да изчислим:

Progress = 7 / 10 = 70%

Remaining = 3

Attendance = 8 / 10 = 80%

Absences = 2

Така няма опасност да запишем:

Attendance = 8 / 10

Percentage = 90%

и после да се чудим кое е вярно.

Значи към критериите на V1 добавяме:

За всеки Student / Студент системата трябва да може да покаже напредъка в курса и присъствието на занятията както като абсолютна стойност (7 / 10), така и като процент (70%).

И още по-важно:

Progress / Напредък и Attendance / Присъствие са две отделни метрики и не трябва да се смесват.

Това според мен вече е **задължителна част от V1.**

35. Вие казахте:

в1 ще е общо за курса, а в2 ще добавим теми и други неща които в в1 са били само прости числа, а тук ще станат изчисляеми

36. ChatGPT каза:

Да — **точно това е логиката**, която според мен трябва да приемем.

V1 — Course / Курс

Във V1 гледаме курса **на общо ниво**.

Тоест не знаем още подробностите за:

- Topic / Тема
- Lesson / Занятие
- конкретни учебни единици
- отделни оценки по занятия/теми
- детайлно присъствие по всяко занятие

Затова във V1 можем да имаме **прости обобщени стойности**.

Например за даден студент:

Course / Курс: C# Основи

Progress / Напредък:

7 / 10

70%

Attendance / Присъствие:

8 / 10

80%

IntermediateGrade / Междинна оценка:

5.25

FinalGrade / Крайна оценка:

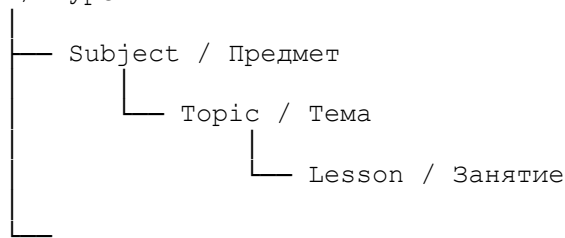
5.75

Тук 7 / 10 и 8 / 10 са **обобщени стойности**.

V2 — Course Details / Детайли на курса

Във V2 вече добавяме реалната структура:

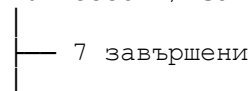
Course / Курс



И тогава числата от V1 вече **не са просто въведени числа**, а могат да се изчисляват от реалните данни.

Например:

10 Lesson / Занятия



└─ 3 оставащи

Системата изчислява:

Progress = 7 / 10 = 70%

А присъствието:

10 проведени занятия

└─ 8 присъствал
└─ 2 отсъствал

и:

Attendance = 8 / 10 = 80%

Това е много важно за архитектурата

V1 определя каква информация искаме да виждаме.

V2 определя откъде тази информация може да бъде изчислена.

Тоест:

V1

Обобщена информация

↓
7 / 10
70%
8 / 10
80%
5.25
5.75

↓

V2

Детайлни данни

↓
Subject

↓
Topic

↓
Lesson

↓
Attendance

↓
Assessment

↓

Calculated / Изчислено

↓
Progress
Attendance
Grades
Remaining
Statistics

Това ни позволява **да не усложняваме V1**, но същевременно да не затваряме архитектурата.

Значи за V1 приемаме следния принцип:

V1 работи с обобщени стойности за курса. V2 добавя детайлната структура и превръща част от тези стойности в изчисляеми показатели на база реалните данни.

Това според мен е много добро разделяне.

И още нещо: когато стигнем до V2, **няма да изхвърляме V1 полетата автоматично**. Ще решим кои остават като реално съхранявани стойности и кои стават изцяло *calculated* / изчисляеми.

37. Вие казахте:

мисля че с това сме готови макар че в процеса на работа ще решат кво още ще трябва и кое да оставим на друга версия

38. ChatGPT каза:

Да, съгласен съм. 👍 Това е нормалният начин да се проектира такава система — **не се опитваме да предвидим абсолютно всичко от първия път**.

Нека тогава считаме **V1 — Course / Курс** за първоначално завършена концепция.

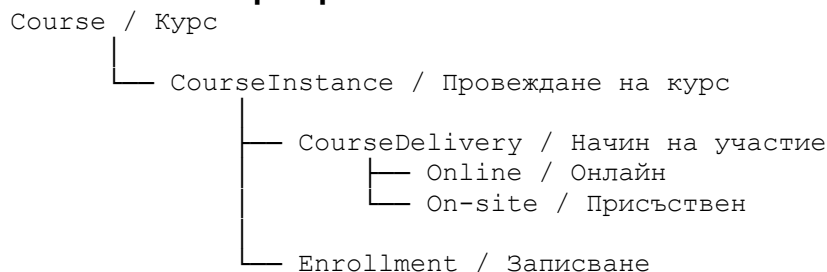
V1 — Course / Курс

Основна идея

Course / Курс е общо учебно предложение, независимо от конкретната област — програмиране, език, математика, бизнес и т.н.

Един курс може да има различни конкретни провеждания.

Основни концепции



Hybrid / Смесен се получава, когато конкретното провеждане има и Online, и On-site вариант.

Информация за студента

За конкретен студент в конкретно провеждане трябва да можем да покажем:

- Status / Статус — записан, обучава се, завършил и т.н.
- Progress / Напредък — например 7 / 10, 70%
- Remaining / Оставашо — например 3 / 10
- Attendance / Присъствие — например 8 / 10, 80%
- IntermediateGrade / Междинна оценка

- FinalGrade / Крайна оценка

В **V1** тези показатели са на **общо ниво**.

В следващите версии, когато добавим Subject / Предмет, Topic / Тема, Lesson / Занятие и други детайли, част от тях ще могат да се изчисляват автоматично.

Важен принцип

V1 не се опитва да предвиди всички бъдещи версии.

Ако по време на разработката установим, че нещо:

- липсва → добавяме го;
- е излишно → махаме го;
- принадлежи към друга концепция → преместваме го;
- изисква по-детайлен модел → оставяме го за следваща V#.

Това означава, че **V1 е първата стабилна основа, а не окончателният модел на цялата система.**

Статус

V1 — Course / Курс → APPROVED 

Следващата концепция можем да започнем като:

V2 — Module / Модул

и там вече ще разгледаме свързването на Module / Модул с Course / Курс чрез свързваща таблица, както решихме.

1. Вие казахте:

остана ни да видим с следваща точка

2. ChatGPT каза:

Да. 👍 V1 е приключена, но **не сме приключили цялата архитектура**. Просто преминаваме към следващата концепция.

V2 — Module / Модул

Тук ще започнем от нулата и ще изясним само **какво представлява Module / Модул** и как се свързва с вече готовия Course / Курс.

Ще използваме същия подход:

1. **Име на модула**
2. **Кратко описание**
3. **Какво съдържа**
4. **Какви са изискванията**

5. **Как се свързва с Course / Курс**
6. **Какви таблици са необходими**
7. **Какви са правилата**
8. Накрая → **APPROVED**

И няма да приемаме предварително, че трябва да е точно определен брой таблици.

V2 — първи въпрос

Какво искаме да означава **Module / Модул** в реалността?

Например:

Module / Модул

"Основи на програмирането"

и в него:

Course / Курс

├─ C# Основи

├─ Python Основи

└─ JavaScript Основи

или:

Module / Модул

"Английски език"

├─ English A1

├─ English A2

└─ Business English

Тук вече можем да приложим решението, което взехме по-рано:

ModuleCourse / Модул-Курс като свързваща таблица, чрез която

определяме **кои курсове участват в даден модул и кои не**.

Но преди да стигнем до таблицата, нека първо определим **какво точно трябва да представлява Module / Модул**.

3. Вие казахте:

сега обсъждаме в1

4. ChatGPT каза:

Да, прав си. Аз прескочих към V2. Връщаме се към **V1 — Course / Курс**.

Щом казваш, че **е останала още една точка за V1**, нека я определим, вместо да приключваме версията преждевременно.

V1 — Course / Курс

Досега уточнихме:

1. какво представлява Course / Курс;
2. CourseInstance / Провеждане на курс;
3. CourseDelivery / Начин на участие;

4. Enrollment / Записване;
5. капацитет за Online / On-site / Hybrid;
6. общ напредък;
7. оставащо обучение;
8. присъствие;
9. междинна оценка;
10. крайна оценка;
11. кои показатели са прости във V1 и кои могат да станат изчисляеми във V2.

Кажете ми коя е останалата точка, която имаш предвид за V1, и продължаваме точно от нея.

5. Вие казахте:

таблица преподавател

6. ChatGPT каза:

Да. Това е логична част от **V1 — Course / Курс**.

Но тук бихме изчистили първо **какво означава преподавателят**, преди да решим таблицата.

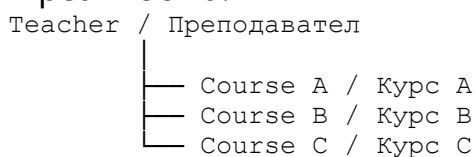
V1 — Teacher / Преподавател

Основна концепция

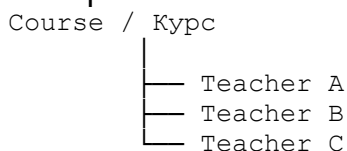
Teacher / Преподавател е човек, който може да води или преподава дадено Course / Курс.

Важно е да не го обвързваме прекалено твърдо с един курс.

В реалността:



И обратното:



Затова естествено се получава **N:M връзка**.

Основна таблица

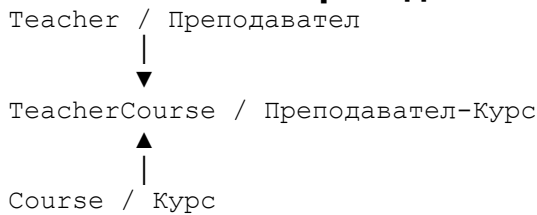
Примерно:

Поле / Field	Пример
Id	1

Поле / Field	Пример
FirstName	Иван
LastName	Петров
Email	ivan@example.com
Phone	+359...
Description	Senior преподавател по програмиране
IsActive	true

Но има нещо, което бихме могли да отделим:

TeacherCourse / Преподавател-Курс



Това позволява:

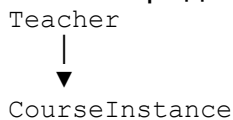
Иван
├─ C# Основи
└─ Python

Мария
├─ C# Основи
└─ JavaScript

И тук идва важният въпрос

Дали преподавателят се определя към **Course / Курс** или към **CourseInstance / Провеждане на курс**?

Аз бих предпочел:



Защото един курс може да се проведе няколко пъти с различни преподаватели.

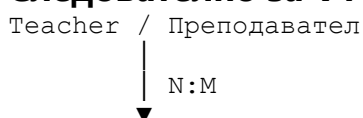
Например:

C# Основи

- ├─ Провеждане 2026
 - └─ Иван
- ├─ Провеждане 2027
 - └─ Мария
- └─ Провеждане 2027/2
 - ├─ Иван
 - └─ Петър

Това е по-реалистично.

Следователно за V1 предлагам



CourseInstance / Провеждане на курс

чрез:

TeacherCourseInstance / Преподавател-Провеждане

И това е ПО-ГЪВКАВО ОТ TeacherCourse.

Ако сме съгласни с тази логика, следва да решим **какви точно полета трябва да има** Teacher / Преподавател. Например дали само лични данни или и квалификация, специализация, опит и т.н.

7. Вие казахте:

това до тук ми харесва

8. ChatGPT каза:

Супер. 👍 Тогава **не го разширяваме излишно** и фиксираме това решение.

V1 — Teacher / Преподавател

Прието решение

Teacher / Преподавател е **самостоятелна концепция**.

Един преподавател може да участва в **МНОГО** CourseInstance /

Провеждане на курс, а едно провеждане може да има един или повече преподаватели.

Teacher / Преподавател

↓
N:M

TeacherCourseInstance / Преподавател-Провеждане

↑
↓

CourseInstance / Провеждане на курс

Пример:

Иван

- └─ C# Основи — Провеждане 2026
- └─ Python — Провеждане 2026
- └─ C# Advanced — Провеждане 2027

Мария

- └─ C# Основи — Провеждане 2027
- └─ JavaScript — Провеждане 2027

Това ни позволява по-късно да правим и статистика:

1. колко курса е водил преподавателят;
2. колко студенти е обучавал;
3. кои курсове е водил;
4. колко провеждания е имал;
5. резултати на студентите и т.н.

Статус: APPROVED 

Следващата част на Teacher / Преподавател е да решим **каква информация пазим за самия преподавател**. И тук бих вървял пак бавно — поле по поле, без да добавяме неща, които може да се окажат ненужни.

9. Вие казахте:

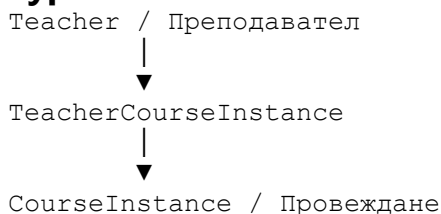
в2 ще предложим и вариант в които коя тема кой я е преподавал

10. ChatGPT каза:

Да, точно така.  И това е добър пример защо **не трябва да претоварваме V1**.

V1 — Teacher / Преподавател

Оставяме връзката на ниво **CourseInstance / Провеждане на курс**:



Тоест във V1 знаем:

Кой преподавател е бил ангажиран с конкретното провеждане на курса.

Не знаем още:

Кой преподавател е преподавал конкретна тема или занятие.

V2 — Детайлно преподаване

Когато въведем:

Subject / Предмет



Topic / Тема



Lesson / Занятие

можем да добавим връзка:

Teacher / Преподавател



TeacherLesson / Преподавател-Занятие



Lesson / Занятие

Тогава ще можем да отчетем например:

C# Advanced

Topic / Тема:

LINQ

Lesson / Занятие:
Занятие №7

Teacher / Преподавател:
Иван Петров

И статистиката вече може да показва:

Иван Петров е преподавал 18 занятия, от които 6 по LINQ, 4 по EF Core и 8 по ASP.NET Core.

Това е **много по-прецизно**, но няма нужда да го вкарваме във V1.

Значи го записваме като бъдещо изискване:

V1:

Teacher / Преподавател → CourseInstance / Провеждане

V2:

Teacher / Преподавател → Lesson / Занятие

и евентуално → Topic / Тема

Така V1 остава проста, а V2 ще може да изгради детайлна статистика.

Тази точка я считаме за приета за V1. 

1. Вие казахте:

график на курса кой дни ще провежда и в колко часа остават в в2

2. ChatGPT каза:

Да, съгласен съм.  **Графикът го оставяме за V2.**

V1 — Course / Курс

Във V1 **не описваме конкретните дни и часове** на занятията.

Тоест не добавяме още:

- Day / Ден
- StartTime / Начален час
- EndTime / Краен час
- периодичност;
- конкретни занятия по дати.

В V1 ни е достатъчно да знаем общата информация за курса и неговото провеждане.

V2 — Schedule / График

Във V2 вече ще можем да кажем например:

Course / Курс: C# Основи

Schedule / График:

Понеделник 18:00 – 20:00

Сряда

18:00 – 20:00

И после, когато добавим Lesson / Занятие, графикът ще може да се свърже с реалните занятия.

Това ще ни позволи да изчисляваме:

- колко занятия са планирани;
- колко са проведени;
- кои са следващите;
- колко остава на студента;
- присъствие;
- натоварване на преподавателя;
- статистика по дни и часове.

Значи за V1 записваме:

Schedule / График → НЕ влиза във V1.

Schedule / График → планиран за V2, заедно с детайлите

на Subject / Предмет, Topic / Тема, Lesson / Занятие И ИЗЧИСЛЯЕМИЯ Progress / Напредък.

Това решение го считаме за прието. 

3. Вие казахте:

само общия брой теми; кой дни ще са занятията -> в1

4. ChatGPT каза:

Да — това променя V1 по добър начин. 

В V1 — **Course / Курс** можем да имаме **обща, опростена информация за организацията на курса**, без още да описваме всяка отделна тема и занятие.

V1 — Course / Курс

Включваме:

Общ брой теми

TotalTopics / Общ брой теми: 10

Тук още **нямаме самите теми**. Просто знаем, че курсът съдържа например 10 теми.

График на дните

Можем да пазим **кои дни от седмицата има занятия:**

CourseDays / Дни на занятия:

- ☒ Понеделник
- ☐ Вторник
- ☒ Сряда

☐ Четвъртък

☐ Петък

Тоест:

Monday / Понеделник

Wednesday / Сряда

Но часовете остават за V2.

Така V1 ще знае:

Course / Курс

Name / Име

Description / Описание

Requirements / Изисквания

Level / Ниво

TotalTopics / Общ брой теми: 10

CourseDays / Дни:

Monday / Понеделник

Wednesday / Сряда

DeliveryMode / Начин:

Online / Онлайн

On-site / Присъствен

Hybrid / Смесен

Capacity / Капацитет:

Online: 20

On-site: 10

А V2 ще превърне:

TotalTopics = 10

в реална структура:

Subject / Предмет

```
├── Topic 1
├── Topic 2
├── ...
└── Topic 10
```

и:

Monday + Wednesday

в реален:

Schedule / График

↓

Lesson / Занятие

↓

Date + StartTime + EndTime

Това е много хубаво разделение:

V1 = обща картина на курса.

V2 = детайли и изчисления на база реалните данни.

Така TotalTopics / Общ брой теми във V1 може да е просто **10**, а във V2

вече системата ще може да изчисли дали студентът е завършил 7 /

10 на база реално завършени теми.

Тази промяна я записваме към V1. 

5. Вие казахте:

отивам се да го направя като визитна карта на курса и някоя обща информация за курса и студентите

6. ChatGPT каза:

Да — **точно това според мен е правилната посока за V1.** 

Вече го виждаме не като „пълната таблица на курса“, а като **визитна карта / обобщен профил на курса.**

V1 — Course / Курс

Идеята

v1 трябва да отговори бързо на въпроса:

„Какво представлява този курс и как върви обучението в него?“

Без да навлизаме още в подробностите на темите, занятията и оценяването.

Визитна карта на курса

Например:

C# Основи	
Код:	CS-001
Описание:	Основи на C#
Ниво:	Начинаещи
Език:	Български
Теми:	10
Дни:	Понеделник, Сряда
Начин:	Hybrid / Смесен
Online:	20
On-site:	10
Студенти:	23 / 30
Активни:	20
Завършили:	3

И вече имаме **обща информация за студентите**, без още да показваме детайлите.

Например:

Students / Студенти

Total:	23
Active:	20
Completed:	3
Cancelled:	0

И можем да имаме обобщени показатели:

Progress / Напредък:

Средно 72%

Attendance / Присъствие:
Средно 84%

Intermediate Grade:
5.32

Final Grade:
5.61

Но тези последните вече трябва внимателно да решим дали са **реално съхранявани** или **изчисляеми обобщения**.

Така V1 има много ясна граница

V1 показва:

- кой е курсът;
- какво представлява;
- основни изисквания;
- ниво;
- език;
- общ брой теми;
- кои дни се провежда;
- начин на провеждане;
- капацитет;
- обща информация за записаните студенти;
- общи показатели за обучението.

V1 НЕ показва още:

- конкретните теми;
- конкретните занятия;
- кой преподавател е водил конкретна тема;
- конкретния график по часове;
- детайлното присъствие;
- детайлното оценяване.

Тези неща могат да се развият във **V2**.

И според мен това вече е много добра дефиниция:

V1 — Course / Курс е визитна карта на курса и обобщена информация за текущото му състояние и студентите в него.

Това е достатъчно конкретно, за да започнем да определяме **таблиците и полетата на V1**, но достатъчно гъвкаво, за да не заключим бъдещите версии.

1. коя година е проведен къде/кой град, ако е присъствено/, вечер/сутрин и т.н

2. ChatGPT каза:

Да — точно тези неща пасват идеално на идеята за **V1 като „визитна карта“ на конкретното провеждане на курса.**

Само бих направил едно разграничение: **курсът и провеждането на курса** не са едно и също.

V1 — Course / Курс

Course / Курс

Това е самият курс:

C# Основи

Описание: Въведение в C#

Ниво: Начинаещи

Език: Български

Общ брой теми: 10

CourseInstance / Провеждане

Тук вече е конкретната „визитна картичка“:

Course: C# Основи

Year / Година:

2026

Location / Място:

Vidin / Видин

Delivery / Начин:

Hybrid / Смесен

Online capacity:

20

On-site capacity:

10

Days:

Понеделник

Сряда

TimePeriod / Период:

Evening / Вечер

Students:

23 / 30

За „сутрин / следобед / вечер“

Аз не бих записвал часове във V1, щом сме решили часовете да останат за V2.

Но можем да имаме общо поле:

TimeOfDay / Част от деня

с примерно:

Morning / Сутрин

Afternoon / Следобед

Evening / Вечер

Това е достатъчно за визитната карта.

По-късно V2 може да го превърне в реален график:

Monday

18:00 – 20:00

Wednesday

18:00 – 20:00

За мястото

Тук също бих го направил универсално.

Ако е присъствено:

City / Град: Видин

Ако е онлайн:

City: –

Ако е смесено:

Online + On-site

City: Видин

По-късно V2 можем да добавим:

Building

Room

Address

ако се окаже необходимо.

Така визитката на конкретното провеждане става примерно:

С# Основи	
Провеждане: 2026	
Година:	2026
Град:	Видин
Начин:	Hybrid
Период:	Вечер
Дни:	Понеделник, Сряда
Теми:	10
Online:	20 места
Присъствено:	10 места
Студенти:	23 / 30

Това вече наистина изглежда като **визитна карта на курса + конкретното му провеждане**.

И според мен сме стигнали до много добра граница на

V1: достатъчно информация, за да знаем какво представлява

и как се провежда курсът, но без да влизаме в детайлната учебна структура на V2.

3. Вие казахте:

детайлите ги оставаме за в2

4. ChatGPT каза:

Да. Точно така го фиксираме. 👍

V1 — Course / Курс

Цел: визитна карта + обща информация за конкретното провеждане и студентите.

Във V1 пазим само обобщена информация:

- име и код на курса;
- описание;
- изисквания;
- ниво;
- език;
- общ брой теми;
- година на провеждане;
- град/място, когато е присъствено;
- начин на провеждане — онлайн / присъствено / смесено;
- капацитет по начин на провеждане;
- дни на занятията;
- част от деня — сутрин / следобед / вечер;
- преподаватели за конкретното провеждане;
- брой и общ статус на студентите;
- за всеки студент — статус, общ напредък, присъствие, междинна и крайна оценка.

V2 — Детайли

Оставаме за следващата версия:

- конкретните теми;
- занятията;
- точните дати и часове;
- детайлния график;

- кой преподавател е водил коя тема/занятие;
- детайлното присъствие;
- детайлното оценяване;
- автоматичното изчисляване на прогреса от реалните теми/занятия;
- по-подробната статистика.

Така **V1 не е бедна версия**, а ясна самостоятелна версия с конкретна цел.

V1 = „Какво е този курс, как се провежда и какво е общото състояние на студентите?“

V2 = „Какво точно се случва вътре в курса?“

Това разделение според мен вече е много чисто. 

5. Вие казахте:

ниво студенти, учители и курсове е завършено

6. ChatGPT каза:

Да.  Приемаме ги като **завършени концепции за V1.**

V1 — Завършени

- **Course / Курс** 
 - Визитна карта на курса
 - Основна информация за провеждането
 - Начин на провеждане и капацитет
 - Дни и част от деня
 - Град/място
 - Обобщена информация за студентите
 - Общ прогрес, присъствие и оценки
- **Student / Студент** 
 - Записан курс
 - Статус
 - Напредък 7/10 и %
 - Оставащо
 - Присъствие 7/10 и %
 - Междинна оценка

- Крайна оценка
- **Teacher / Преподавател** 
 - Самостоятелна концепция
 - Връзка с конкретно провеждане на курс
 - Един курс може да има повече преподаватели
 - Един преподавател може да води различни курсове

V1 принцип

Общата картина е готова.

Детайлите остават за следващите версии.

V1

— Student / Студент	
— Teacher / Преподавател	
— Course / Курс	

Следващата точка вече можем да я определим като **следваща основна концепция на V1**, без да се връщаме постоянно към тези три, освен ако по време на работата не открием нещо, което трябва да променим.

1. Вие казахте:

може ли да видим до къде сме с нашия план и архитектура на системата - може ли обобщение до сега

2. ChatGPT каза:

Да. И всъщност вече имаме **доста ясна основа**. Ето обобщението на това, което сме решили досега, без да добавям неща, които още не сме обсъждали.

Архитектура на системата — текущо състояние

Основен принцип

Системата няма да бъде направена само за програмиране или конкретна област.

Тя трябва да може да се използва за:

- програмиране;
- езици;
- математика;
- бизнес;

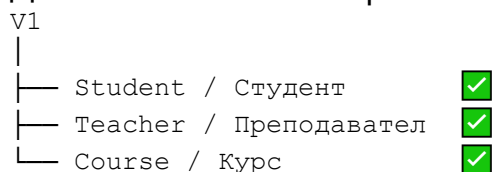
- други видове обучения.

Тоест **областта на обучение не е част от основната архитектура.**

V1 — Основна версия

V1 представлява **общата картина на обучителната система.**

До момента сме завършили три основни концепции:



1. Student / Студент ✓

За студента трябва да можем да виждаме какво се случва с неговото обучение по конкретен курс.

Информация за курса

Студентът може да бъде:

- записан;
- обучаващ се;
- завършил;
- отказал се/прекратил.

Progress / Напредък

Във V1 е обобщена стойност:

7 / 10
70%

и можем да покажем:

Remaining:
3 / 10

Attendance / Присъствие

Отделно от прогреса:

7 / 10
70%

Това е важно:

Progress $\neq$ Attendance

Студентът може например да има:

Progress: 70%
Attendance: 90%

Оценки

Пазим:

Intermediate Grade / Междинна оценка
Final Grade / Крайна оценка

Във V1 те са **обобщени стойности**.

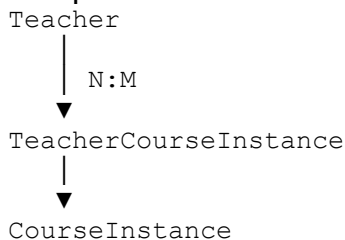
В V2 ще можем да изградим детайлна система за оценяване и част от тези стойности да станат изчисляеми.

2. Teacher / Преподавател

Преподавателят е самостоятелна концепция.

Не го обвързваме директно с един курс.

Избрахме:



Тоест:

- о един преподавател може да води много провеждания;
- о едно провеждане може да има повече от един преподавател.

Пример:

```
Иван
├── C# - 2026
├── Python - 2026
└── C# Advanced - 2027
```

```
Мария
├── C# - 2027
└── JavaScript - 2027
```

Какво НЕ правим във V1

Не записваме още:

„Иван е преподавал тема LINQ.“

Това остава за V2, когато въведем теми и занятия.

3. Course / Курс

Тук направихме най-много уточнения.

Основната идея е:

V1 Course е визитна карта на курса + обща информация за конкретното му провеждане и студентите.

Course / Курс

Самият курс съдържа обща информация:

Name / Име

Code / Код
Description / Описание
Requirements / Изисквания
Level / Ниво
Language / Език
TotalTopics / Общ брой теми

Във V1 не описваме самите теми.

Например:

C# Основи

Total Topics: 10

Но не:

1. Variables
2. Classes
3. LINQ
- ...

Това е V2.

CourseInstance / Провеждане

Разделихме **Course** от конкретното му провеждане.

Например:

Course:
C# Основи

↓

CourseInstance:
C# Основи — 2026

Така един курс може да бъде провеждан многократно.

Например:

C# Основи

- Провеждане 2026
- Провеждане 2027
- Провеждане 2028

И всяко провеждане може да има различна организация.

Начин на провеждане

Имаме три основни възможности:

Online / Онлайн
On-site / Присъствен
Hybrid / Смесен

Но взехме важно решение:

Hybrid не е задължително отделен специален тип данни.

При смесено обучение можем да имаме:

Online
Capacity: 20

On-site

Capacity: 10

и системата разбира, че това е Hybrid.

Това позволява статистика по начин на участие.

Capacity / Капацитет

При смесено обучение потребителят може да определи отделни капацитети:

Online: 20

On-site: 10

Total: 30

Това е по-добре от:

HybridCapacity: 30

защото знаем **как са разпределени местата**.

Дни и период на деня

Във V1 решихме да пазим:

Дни

Например:

Monday / Понеделник

Wednesday / Сряда

Част от деня

Morning / Сутрин

Afternoon / Следобед

Evening / Вечер

Но **точните часове** остават за V2.

Година и място

За конкретното провеждане във V1 можем да знаем:

Year:

2026

City:

Vidin / Видин

ако обучението е присъствено или смесено.

Не навлизаме още в:

Building

Room

Address

освен ако по време на реализацията не решим, че са необходими.

Какво нарочно оставихме за V2

Това е много важно, защото вече имаме ясна граница между версиите.

V2 — Детайли на обучението

Ще добавим:

Subject / Предмет

↓

Topic / Тема

↓

Lesson / Занятие

Тогава например:

Total Topics = 10

от V1 ще се превърне в реални 10 теми.

И:

Progress = 7 / 10

вече може да бъде **изчислен**, вместо просто въведен.

V2 — Детайлен график

Във V1:

Monday

Wednesday

Evening

Във V2:

Monday

18:00 - 20:00

Wednesday

18:00 - 20:00

и конкретните занятия могат да имат реални дати и часове.

V2 — Детайлно преподаване

Във V1:

Teacher → CourseInstance

Във V2 можем да имаме:

Teacher → Lesson

и да знаем:

кой преподавател е водил конкретната тема/занятие.

V2 — Детайлно присъствие

Във V1:

Attendance:

7 / 10

70%

Във V2:

Lesson 1 → Present

Lesson 2 → Present

Lesson 3 → Absent

...

и системата изчислява:

7 / 10

70%

V2 — Детайлни оценки

Във V1:

Intermediate: 5.25

Final: 5.75

Във V2 можем да имаме:

Quiz

Assignment

Practical exam

Midterm

Final exam

и съответните оценки да се използват за изчисляване на
междинната и крайната оценка.

Най-важният архитектурен принцип, който излезе

Мисля, че това е едно от най-силните ни решения досега:

V1 описва общото състояние. V2 добавя реалните детайли, от които това състояние може да бъде изчислено.

Например:

V1

Progress: 7 / 10

Attendance: 8 / 10

↓

V2

10 реални теми

10 реални занятия

8 присъствия

7 завършени

↓

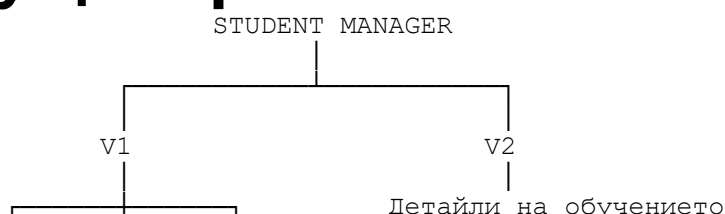
Calculated

Progress: 70%

Attendance: 80%

Remaining: 3

Текуща карта



Да. Ако гледаме **само V1**, трите основни области ги имаме. Остава да изчистим няколко архитектурни въпроса, преди да кажем, че **V1 е напълно готова за проектиране на базата**.

Какво остава за V1

1. User / Потребител

Това според мен е най-важната липсваща концепция.

Трябва да решим кой влиза в системата и какви роли има:

- Student / Студент
- Teacher / Преподавател
- Administrator / Администратор
- евентуално други

И най-вече:

Student и Teacher отделни потребители ли са, или са профили/роли на един User?

Това е важно за цялата архитектура.

2. Enrollment / Записване

Обсъдихме го като концепция, но още не сме го изчистили като самостоятелна част.

Трябва да решим:

- кой студент се записва;
- в кой курс;
- в кое конкретно провеждане;
- кога е записан;
- какъв е статусът му;
- Online или On-site;
- какво става при отказване/завършване.

3. CourseInstance / Провеждане на курс

Знаем какво искаме да съдържа, но още не сме го разгледали **поле по поле**.

Тук са:

- година;
- град;
- начин на провеждане;

- капацитет;
- дни;
- сутрин/следобед/вечер;
- преподаватели;
- студенти.

Това вероятно ще е една от централните таблици.

4. Статусите

Трябва да определим окончателно какви статуси използваме.

Например за студент:

Enrolled
In Progress
Completed
Cancelled

Но трябва да решим дали това са всички.

5. Изисквания за курс

Споменахме Requirements / Изисквания, но още не сме решили дали ще бъде:

едно текстово поле

или:

отделна структура

За V1 аз бих започнал максимално просто, но трябва да го решим.

6. Език

Имаме Language / Език, но трябва да решим дали V1 позволява:

Bulgarian
English
...

и дали курсът може да има повече от един език.

7. Нива

Споменахме Level / Ниво, но още не сме определили какво означава:

Beginner
Intermediate
Advanced

Дали са фиксирани или администраторът може да създава свои.

8. История и архив

Трябва да решим какво става с:

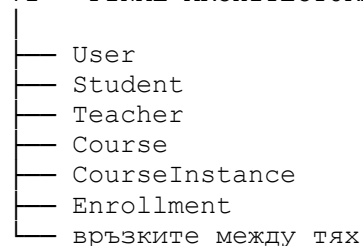
- стар курс;
- старо провеждане;
- неактивен преподавател;
- завършил студент.

По-скоро **не трябва да изтриваме историческите данни**, защото после статистиката ще се счупи.

А след това

Когато тези неща са изчистени, можем да направим:

V1 — FINAL ARCHITECTURE



и чак тогава да започнем **реалните таблици с полета, типове и Foreign Keys**.

Моето предложение за следваща точка

Да започнем с **User / Потребител**, защото той е основата, върху която могат да стъпят Student / Студент, Teacher / Преподавател и евентуално Administrator / Администратор.

Това ще ни помогне и да избегнем много сериозен архитектурен проблем по-късно — **един човек да бъде едновременно студент и преподавател**, ако някога се наложи.

1. може ли да ги направим на нива/роли
2. ChatGPT каза:

Да — и според мен **ролите са по-добрият вариант**. Така не заключваме един човек само в една категория.

User / Потребител

Правим една основна концепция:

User / Потребител

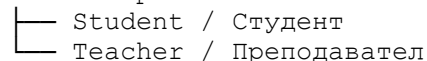


Role / Роля

Един потребител може да има **една или повече роли**.

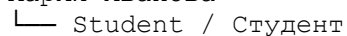
Например:

Иван Петров



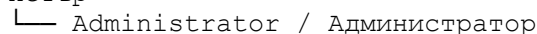
Друг:

Мария Иванова



А администратор:

Петър



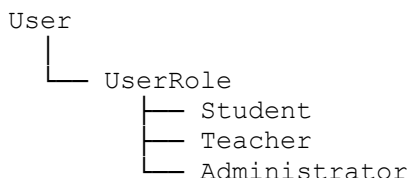
Предимството

Не правим:

```
UserType = Student
```

защото това би означавало **само една роля**.

Вместо това:



Така един човек спокойно може да бъде:

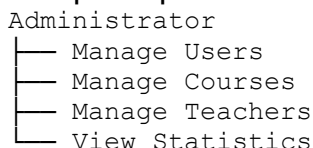
Student + Teacher

без да създаваме два отделни акаунта.

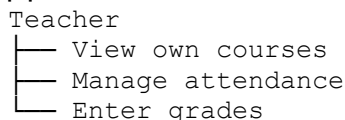
И можем да имаме нива на достъп

Тук бих разделил **Role / Роля** от **Permission / Разрешение**.

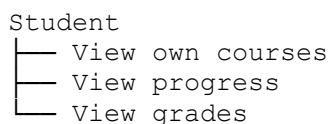
Например:



Докато:



И:



Това вече ни дава **Role-Based Access Control (RBAC)**.

За V1 обаче бих го направил постепенно

Първо да определим:

User → Roles

и кои са основните роли.

После можем да решим дали **Permissions / Разрешения** влизат във V1 или ги оставяме за следваща версия.

Така не усложняваме системата прекалено рано.

Следващият въпрос е: кои роли искаме да имаме в V1?

1. 4 роли: админ; студент; преподавател; ит
2. ChatGPT каза:

Да. Имаме **4 роли**. Само последната не си я довършил — написа „ИТ“. 😊

Засега:

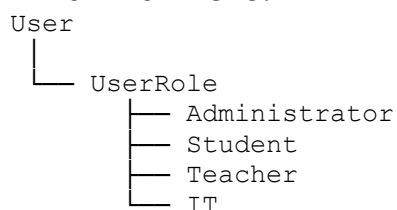
V1 — Roles / Роли

1. Administrator / Администратор
2. Student / Студент
3. Teacher / Преподавател
4. IT / ИТ

Ако под **ИТ** имаш предвид технически потребител/системен администратор, бих го разграничил от Administrator / Администратор. Например:

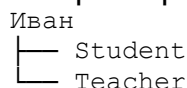
- **Administrator** → управлява учебната част на системата — студенти, преподаватели, курсове и т.н.
- **ИТ** → управлява техническата част — системата, настройки, достъп, технически проблеми и др.

И най-важното:

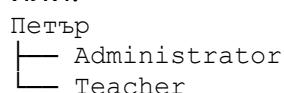


Един User може да има **повече от една роля**.

Например:



или:



Така архитектурата остава гъвкава.

Само потвърди какво точно имаш предвид под четвъртата роля „ИТ“, за да я запишем окончателно.

3. Вие казахте:

системен администратор

4. ChatGPT каза:

Да, така вече е ясно. 👍

V1 — User / Потребител и Roles / Роли

Приемаме **4 основни роли**:

1. Administrator / Администратор
2. Student / Студент
3. Teacher / Преподавател

4. System Administrator / Системен администратор

Разлика между Administrator и System Administrator Administrator / Администратор

Отговаря за **учебната и организационната част**:

- студенти;
- преподаватели;
- курсове;
- записвания;
- оценки;
- присъствие;
- статистики.

System Administrator / Системен администратор

Отговаря за **техническата част**:

- потребители и достъп;
- системни настройки;
- техническа конфигурация;
- сигурност;
- системни логове;
- backup/restore;
- техническа поддръжка.

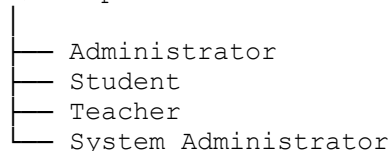
Това разделение е много полезно, защото не всеки администратор на обучението трябва да има достъп до техническите настройки.

Основен модел

User / Потребител



UserRole / Потребител-Роля



И запазваме решението:

Един потребител може да има повече от една роля.

Така например един човек може да бъде едновременно:

Student + Teacher

без два акаунта.

Тази част можем да я считаме за приета за V1. ✓

Следващото, което според мен трябва да изясним, е **какви права има всяка от тези 4 роли**. Това е различно от самата роля и ще ни даде основата за достъпа до системата.

5. Вие казахте:

много се замислям да оставим System Administrator - някой от нещата за v2

6. ChatGPT каза:

Да — аз също бих го оставил за **V2**, освен ако още в V1 нямаме реална нужда от отделен системен администратор.

Причината е проста: в момента V1 изграждаме основно **учебната част на системата**, а System Administrator / Системен администратор е по-скоро инфраструктурна и техническа роля.

V1 — Роли

Оставяме само:

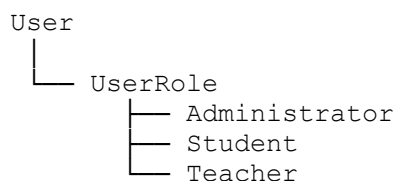
1. Administrator / Администратор
2. Student / Студент
3. Teacher / Преподавател

И:

System Administrator / Системен администратор

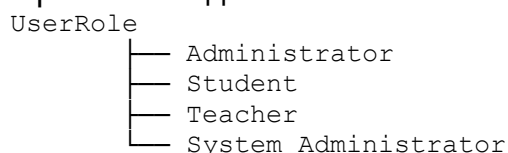
→ **V2**

V1



V2

При необходимост:



Това е и в духа на досегашната ни архитектура:

Не добавяме нещо само защото може да потърбва. Добавяме го, когато имаме реална нужда от него.

И още нещо — това **не означава, че архитектурата V1 няма да позволява добавянето му**. Напротив, ако направим User → UserRole правилно, добавянето на нова роля във V2 ще бъде сравнително лесно.

Значи засега: System Administrator → V2. ✓

7. Вие казахте:

но бих добавил и настройки на системата като:

8. ChatGPT каза:

Да, това е разумно. **Настройки на системата** могат да останат във V1, дори ако самата роля System Administrator остане за V2.

Тук е важно да разделим двете неща:

- **System Settings / Системни настройки** → можем да ги имаме във V1.
- **System Administrator / Системен администратор** → отделна роля за V2.

Например във V1 можем да имаме обща секция:

Settings / Настройки

и вътре да определим какво реално ни трябва.

Кажете какви настройки имаш предвид и ще ги разгледаме **една по една**, както правихме с курса.

9. Вие казахте:

под настройки визирах да направим неща по UI

10. ChatGPT каза:

Ааа, да — разбрах те. 👍 Говориш за **настройки на интерфейса (UI)**, а не за техническа администрация на системата.

Това според мен спокойно може да бъде **част от V1**.

V1 — UI Settings / Настройки на интерфейса

Идеята е потребителят да може да настрои как изглежда и се държи интерфейсът.

Например:

- **Theme / Тема** — светла / тъмна;
- **Language / Език** — български / английски;
- **Layout / Подредба** — ако имаме повече от един вариант;
- **Sidebar / Странично меню** — отворено / свито;
- размер на текста, ако решим, че е необходимо;
- други визуални настройки.

Важно разграничение

Това не е:

System Administrator

и не е:

System Configuration

Това е:

```
User
└── UI Settings
```

Тоест **всеки потребител може да има свои настройки**.

Например:

Иван
Theme: Dark
Language: Bulgarian
Sidebar: Collapsed

а:

Мария
Theme: Light
Language: English
Sidebar: Expanded

И това не влияе на останалите потребители.

В архитектурата

Бих го мислил така:

```
User / Потребител
├── Roles / Роли
└── UI Settings / UI Настройки
```

Това е доста чисто.

И дори ако по-късно добавим System Administrator, **UI Settings няма да зависят от него**.

Така че можем да кажем:

V1: User + Roles + Personal UI Settings

V2: System Administrator + System-level Settings

Ако това е идеята ти, можем сега да определим **какви точно UI настройки искаме във V1**, една по една.

11. Вие казахте:

някой от тези настройки който спомена да се развият в в2 и да има избор от избор в в2 коя избира дали ще е default, user или друга

12. ChatGPT каза:

Да — и това вече е **много добра архитектурна идея**. Така няма да направим настройките прекалено прости във V1, но ще оставим място за сериозно развитие във V2.

V1 — UI Settings / Настройки на интерфейса

Във V1 можем да имаме **основни лични настройки**:

- Theme / Тема
- Language / Език
- Sidebar / Странично меню
- евентуално размер на текста

Пример:

User: Иван

Theme: Dark

Language: Bulgarian

Sidebar: Collapsed

Това е **настройка на конкретния потребител**.

V2 — Settings hierarchy / Нива на настройките

Тук вече можем да направим много по-интересна система.

Една настройка може да има източник:

Default / По подразбиране

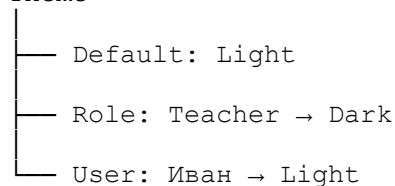
User / Потребителска

Role / Ролева

System / Системна

Например:

Theme



Системата определя коя стойност да използва според приоритета.

Например:

User setting



Role setting



System setting



Default

или друг приоритет, който ще решим във V2.

Пример

Администраторът задава:

Default Theme = Light

За преподавателите:

Teacher Theme = Dark

И конкретен преподавател може да избере:

My Theme = Light

Тогава:

Student:

Light

Teacher:
Dark

Teacher Иван:
Light

Това е много по-гъвкаво.

И точно тук бих предложил едно важно правило

Във V2 нека потребителят не избира ръчно „Default / User / Role“ като обикновена стойност.

По-добре системата да има концепция:

Setting Scope / Обхват на настройката

например:

Default
System
Role
User

и архитектурата да определя **коя стойност има предимство**.

Така няма да объркаме:

„Каква е моята тема?“

с:

„На какво ниво е зададена тази тема?“

Така бих го записал в плана

V1:

Основни персонални UI настройки за потребителя.

V2:

Разширена система за настройки с различни нива на прилагане — Default, System, Role и User — и механизъм за приоритет между тях.

Това е хубава функционалност за V2, защото може да се използва не само за Theme, а и за много други настройки по-късно.

Така и System Administrator ролята може естествено да се появи във V2, когато вече имаме нужда от системно ниво на настройки.

1. Вие казахте:

донякъде се покрива, но има и няколко разлики

2. ChatGPT каза:

Да, точно — има **припокриване**, но не са едно и също. И е добре да не ги смесваме още на ниво архитектура.

Аз бих ги разделил на две понятия:

1. UI Settings / Настройки на интерфейса

Това са настройки **как конкретният потребител предпочита да вижда системата.**

Например:

- тъмна/светла тема;
- език;
- меню;
- размер на елементите;
- подредба;
- други визуални предпочитания.

2. Configuration / Конфигурация

Това са настройки, които определят **как системата работи или какво е разрешено.**

Например:

- кои функции са активни;
- кои полета се показват;
- ограничения;
- системни параметри;
- настройки за конкретна роля.

Тук вече може да има:

Default
Role
User
System

и правила за приоритет.

Къде се припокриват?

Например:

Theme = Dark

може да бъде просто:

User UI preference

но може и да бъде:

Default UI setting

която потребителят може да промени.

Затова не бихме казали:

„Всички настройки са User Settings.“

По-добре да имаме обща концепция **Settings / Настройки**, но различни типове/нива.

И тук мисля, че е по-добре **да не решаваме предварително архитектурата**, а да извадим твоите разлики една по една.

Кажи ми кои са тези няколко разлики, които имаш предвид, и ще ги подредим така, че да не се получи объркване между **UI настройки, настройки на потребителя и системна конфигурация**.

3. Вие казахте:

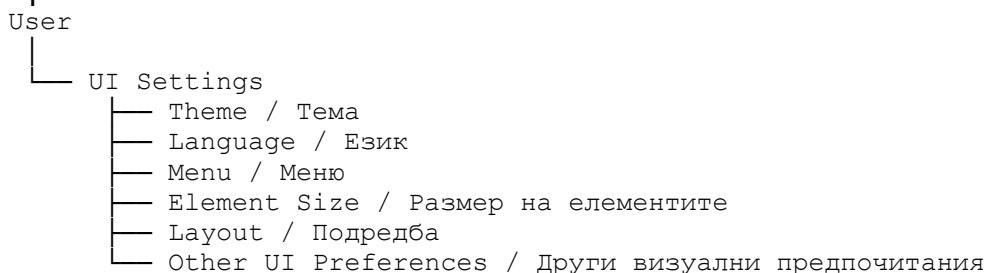
потребителя сам определя настройките си: тъмна/светла тема; език; меню; размер на елементите; подредба; други визуални предпочитания и т.н.

4. ChatGPT каза:

Да — това вече е ясно. И бих го **фиксира**л точно така за **V1**.

V1 — User UI Settings / Лични UI настройки

Тези настройки са **изцяло лични настройки на потребителя**. Потребителят сам определя как иска да изглежда интерфейсът при него:



Например:

Иван Петров

Theme: Dark
Language: Bulgarian
Menu: Collapsed
Element Size: Medium
Layout: Compact

Друг потребител може да има:

Мария Иванова

Theme: Light
Language: English
Menu: Expanded
Element Size: Large
Layout: Comfortable

Настройките на Иван не влияят на Мария.

А V2?

Там можем да развием идеята с различни нива:

Default
↓
System
↓
Role
↓
User

Но това **не променя основната концепция на V1.**

V1 казва:

„Всеки потребител може сам да персонализира своя интерфейс.“

V2 може да добави:

„Системата може да има настройки по подразбиране, настройки за роли и системни настройки, като потребителската настройка може да има предимство.“

Така не смесваме двете неща и запазваме възможност за развитие.

User UI Settings → V1 

5. Вие казахте:

настройки по подразбиране -> ще са ако някой иска да му помогне със системата

6. ChatGPT каза:

Да — това вече изяснява разликата. 

Ти имаш предвид, че **Default / Настройки по подразбиране не са „системни настройки“**, а са **готов профил/начална конфигурация на интерфейса**, който може да помогне на потребител, който не знае как да си го настрои.

V1 — Default UI Settings

Например нов потребител влиза за първи път:

```
Default
├─ Theme: Light
├─ Language: Bulgarian
├─ Menu: Expanded
├─ Element Size: Medium
└─ Layout: Standard
```

Той може:

- да използва тези настройки;
- да ги промени;
- да си направи собствена конфигурация.

Тоест:

Default Settings
↓

User enters
↓
User Settings
↓
User може да промени всичко

Има още един интересен случай

Ако някой помага на потребителя да настрои системата, например преподавател/администратор му показва: „Избери тази подредба, тъмна тема и това меню.“ потребителят може да започне от **Default** и след това да направи собствените си настройки.

Това е различно от:

„Администраторът определя какво трябва да виждаш.“

Това **не искаме във V1**.

Значи имаме две различни неща

Default UI Settings

→ начални настройки, които помагат на потребителя.

User UI Settings

→ личните настройки, които потребителят сам избира.

И:

Default

↓
User

като потребителят може да промени стойностите.

А идеята за **Role/System settings** и по-сложна система за наследяване можем да оставим за V2.

Това според мен вече е много по-точно от първоначалното ми разбиране.

7. Вие казахте:

като и двете ги запазваме и той ги превключва

8. ChatGPT каза:

Да — **сега вече разбирам точно идеята ти.** 👍

Не говорим за:

Default → после се копира в User и Default изчезва.

А за **две налични конфигурации**, между които потребителят може да превключва.

V1 — UI Settings

Имаме две настройки/режима:

UI Settings
○ Default

- My Settings

Default / По подразбиране

Готовата конфигурация:

Theme: Light
Language: Bulgarian
Menu: Expanded
Element Size: Medium
Layout: Standard

Тя остава запазена.

User / Моите настройки

Потребителят си прави собствена:

Theme: Dark
Language: Bulgarian
Menu: Collapsed
Element Size: Large
Layout: Compact

И може да превключва:

Default



My Settings

Тоест **не губи нито една от двете конфигурации.**

И това ни дава хубава възможност за V2

По-късно можем да добавим още режими/профили:

Default
My Settings
Teacher Profile
Student Profile
Custom Profile
...

или:

Default
User
Role
System

Но V1 започва само с:

Default + User Settings

и потребителят решава **коя да използва в момента.**

Това вече бих го записал като конкретно архитектурно решение за V1. 

1. системен администратор си остава и тук идва разлика във версиите: в1 ако той работи система ще показва ще в момента се обновява или нещо подходящо
2. ChatGPT каза:

Да — това вече е **много добра разлика между V1 и V2**. И според мен има **СМИСЪЛ** System Administrator / Системен администратор да остане още във V1, но с **ограничена роля**.

V1 — System Administrator / Системен администратор

Във V1 той не получава цялата бъдеща функционалност за системна администрация.

Основната му задача е системата да може да покаже на останалите потребители:

„Системата в момента се обслужва/обновява.“

Например при вход:

⚙ System maintenance

Системата в момента се обновява.

Някои функции може временно да не са достъпни.

Моля, опитайте отново след приключване на обновяването.

Или по-кратко:

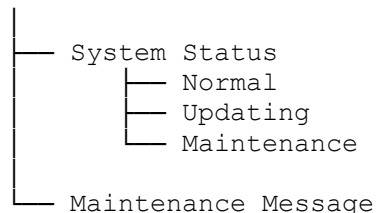
🟡 System status:

System is being updated.

Какво прави System Administrator във V1?

Минимално:

System Administrator



Тоест той може да зададе примерно:

Status: Updating

Message:

"Системата се обновява. Очаквано приключване: 22:00."

а потребителите виждат съобщението.

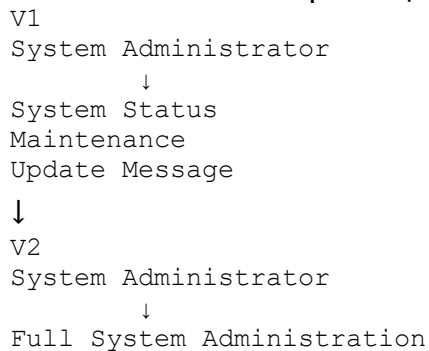
V2 — System Administrator

Тук вече можем да разширим ролята сериозно:

- системна конфигурация;
- управление на системните настройки;
- backup/restore;
- системни логове;

- мониторинг;
- управление на услуги;
- технически настройки;
- по-детайлни съобщения;
- maintenance windows;
- други административни функции.

Така имаме ясна граница:



И това е хубаво, защото **ролята съществува още във V1**, но не е нужно да изграждаме цялата система за системна администрация още сега.

Значи ролите във V1 вече са:

1. Administrator / Администратор
2. Student / Студент
3. Teacher / Преподавател
4. System Administrator / Системен администратор

System Administrator → V1 с базов System Status; V2 с пълна системна администрация. ✓

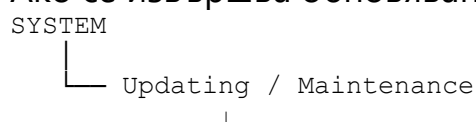
1. V2 - ще са отделни модули които ако един има проблем или се работи по нея или обновява другите модули ще работят не спират работа
2. ChatGPT каза:

Да — това вече е **съществена архитектурна разлика между V1 и V2**.

В V1 можем да мислим за системата като за **едно цяло приложение**, докато във V2 започваме да я разделяме на независими модули.

V1 — System Status

Ако се извършва обновяване или поддръжка:

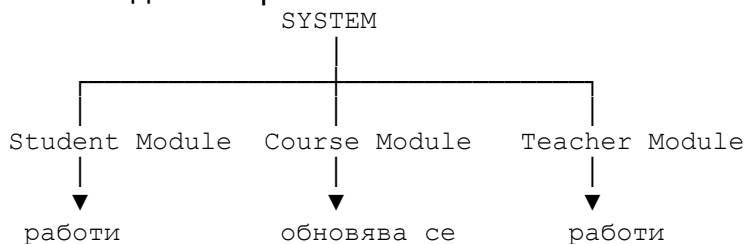


Потребителите получават
подходящо съобщение

Тоест системата може временно да бъде недостъпна или част от функционалността да не работи.

V2 — Modular Architecture / Модулна архитектура

Тук вече идеята е различна:



Например:

Course Module



Updating...

НО:

Student Module	→ Работи
Teacher Module	→ Работи
Reports Module	→ Работи

Така **проблем или обновяване на един модул не трябва автоматично да спира цялата система.**

Това е много важно

Тук вече говорим не просто за „повече таблици“, а за **архитектурно разделяне на системата.**

V2 може да ни даде:

- независими модули;
- независимо обновяване;
- изолиране на проблеми;
- различен статус на всеки модул;
- възможност един модул временно да е недостъпен;
- останалата система да продължи да работи.

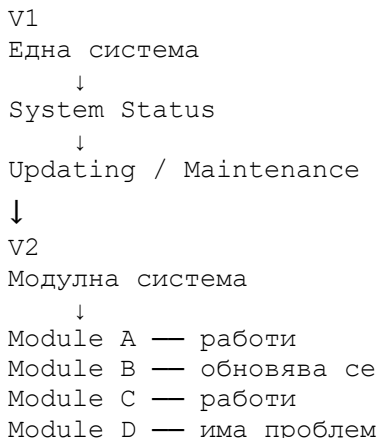
Например потребителят вижда:

System Status

● Students	Operational
● Teachers	Operational
● Courses	Updating
● Reports	Operational

И системата **не му показва общо „System Offline“**, когато всъщност само един модул се обновява.

Така вече имаме ясна еволюция



И тук бих добавил едно важно правило за архитектурата:

V2 не трябва просто да раздели интерфейса на модули.

Модулите трябва да могат да бъдат управлявани и отказват независимо, доколкото архитектурата позволява.

Това ще ни даде много по-стабилна основа за бъдещото развитие.

Тази идея я записваме като основен принцип за V2. 

3. Вие казахте:

или V3

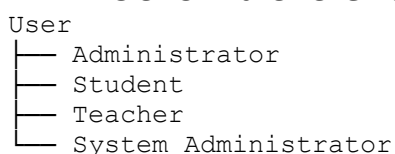
4. ChatGPT каза:

Да — и според мен **дори е по-разумно да го запишем като V3**, а не да се задължаваме, че трябва да го направим във V2.

Защото това вече е **голяма архитектурна промяна**, а не просто добавяне на функционалност.

Версиите започват да се оформят така

V1 — Основна система



Course
Student
Teacher
Enrollment
UI Settings
System Status

Основната цел:

Работеща цялостна система за управление на обучения.

При обновяване/поддръжка можем да показваме:

- Системата се обновява.

V2 — Детайлно обучение

Тук развиваме самото обучение:

Course



Subject



Topic



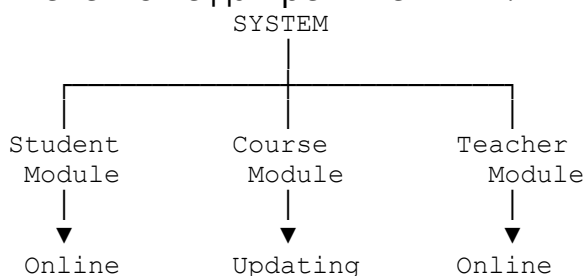
Lesson

Добавяме:

- детайлен график;
- конкретни дати и часове;
- кой преподавател води конкретно занятие;
- детайлно присъствие;
- детайлни оценки;
- изчисляем прогрес;
- по-подробна статистика;
- по-развити настройки.

V3 — Независими модули

Тук вече може да преминем към:



Целта:

Проблем, обновяване или поддръжка на един модул да не спира останалите модули.

И тогава System Administrator може да има много по-сериозна роля за управление на отделните модули.

Това ми харесва повече

Защото не казваме:

„V2 задължително трябва да стане модулна архитектура.“

А казваме:

V2 ще развие функционалността. След като натрупаме достатъчно реален опит със системата, ще преценим дали V3 трябва да премине към независими модули.

Това е много по-здравословно за проекта.

Значи засега:

V1 → основна работеща система 

V2 → детайлизиране и развитие на функционалността 

V3 → евентуална независима модулна архитектура 

И най-важното — **V3 не я заключваме като задължителна.**

Записваме я като архитектурна посока, която ще оценим след V2.

5. Вие казахте:

това ми е за системния администратор кво би добавил или отхвърлил до момента

6. ChatGPT каза:

Да. Ако гледаме **само ролята** System Administrator / Системен администратор, според мен досега сме я очертали добре. Аз бих направил само няколко корекции, за да не я натоварим прекалено рано.

System Administrator — V1

Оставям

1. System Status / Състояние на системата

Например:

 Системата работи нормално

или:

 Системата се обновява

или:

 Системата временно не работи

2. Maintenance / Поддръжка

Системният администратор да може да активира режим, при който системата показва подходящо съобщение.

Например:

Системата в момента се обновява.

Моля, опитайте отново по-късно.

3. Maintenance Message / Съобщение при поддръжка

Това бих го оставил отделно от самия статус.

Например:

Status:

Updating

Message:

„Извършва се планирано обновяване.“

Какво НЕ бих добавял във V1

Backup / Restore ❌ → V2/V3

Важно е, но няма отношение към първоначалната функционалност на системата.

System Logs / Системни логове ❌ → V2

Ще са необходими, но когато имаме повече реална функционалност.

Monitoring / Мониторинг ❌ → V2/V3

CPU, RAM, база данни, услуги и т.н. вече са инфраструктурна тема.

Управление на отделни модули ❌ → V3

Това е свързано директно с идеята ти за независими модули.

Независимо обновяване на модули ❌ → V3

Не бих го залагал като V1/V2 изискване.

Но бих добавил още едно нещо във V1

System Announcement / Системно съобщение

Това е малко по-различно от съобщението при поддръжка.

Например системният администратор може да покаже:

 Внимание

В неделя между 02:00 и 03:00
ще бъде извършена профилактика на системата.

Тоест не е задължително системата **в момента** да е в maintenance.

Може да има:

```
System Status
├── Current Status
├── Maintenance
└── Announcements
```

Това според мен е полезно още от V1.

Моето предложение за V1

Бих ограничил System Administrator ДО:

```
System Administrator
├── System Status
└── Maintenance Mode
```

- Maintenance Message
- System Announcement

И толкова.

А после:

V2

- System Logs
- Monitoring
- по-разширена конфигурация

V3

- Module Management
- Module Status
- Independent Updates
- Fault Isolation

Така ролята **съществува още във V1**, но не превръщаме проекта в система за управление на инфраструктура.

Аз бих запазил всичко, което сме измислили досега, и бих добавил само System Announcement / Системно съобщение.

7. Вие казахте:

приемам

8. ChatGPT каза:

Прието. 

System Administrator / Системен администратор — V1

Фиксираме:

- **System Status / Състояние на системата**
- **Maintenance Mode / Режим на поддръжка**
- **Maintenance Message / Съобщение при поддръжка**
- **System Announcement / Системно съобщение**

V2 / V3

Оставяме за по-късно:

- System Logs / Системни логове → V2
- Monitoring / Мониторинг → V2/V3
- разширена системна конфигурация → V2
- управление на независими модули → V3
- независими обновявания и изолиране на проблеми → V3

Така **System Administrator** за **V1** е достатъчно функционален, но не прекалено сложен.

Статус: **APPROVED** 

9. Вие казахте:

остават другите 3 роли

10. ChatGPT каза:

Да.  System Administrator / Системен администратор ГО изчистихме. Остават трите основни роли:

V1 — Roles / Роли

1. Administrator / Администратор
2. Student / Студент
3. Teacher / Преподавател

И тук предлагам да не ги смесваме с вече обсъдените таблици.

Сега да определим **какво може да прави всяка роля**, а след това ще решим как това се отразява на архитектурата.

1. Administrator / Администратор

Това е ролята, която управлява **учебната част на системата**.

Трябва да решим дали може:

- да създава/редактира/деактивира курсове;
- да управлява студенти;
- да управлява преподаватели;
- да създава провеждания;
- да записва студенти;
- да управлява капацитет;
- да вижда статистика;
- да управлява оценки и присъствие;
- да вижда/променя други данни.

2. Teacher / Преподавател

Основният въпрос е:

Какво може да прави преподавателят със **своите** курсове и студенти?

Например:

- вижда своите провеждания;
- вижда записаните студенти;

- във V1 вижда общия прогрес;
- въвежда/променя оценки;
- отбелязва присъствие;
- вижда статистика за своето провеждане.

Но дали може да **редактира самия курс** — това трябва да решим.

3. Student / Студент

Тук според досегашната ни логика:

- вижда своите курсове;
- вижда статуса си;
- вижда прогреса;
- вижда оставащото;
- вижда присъствието;
- вижда междинната оценка;
- вижда крайната оценка;
- променя собствените си UI настройки.

И трябва да решим дали студентът може да прави още неща — например **сам да се записва за курс**.

Предлагам да ги караме точно както досега

Не искам да ти налагам готова система от права.

Започваме с **Administrator / Администратор** и ти казваш какво според теб трябва да може да прави. После го изчистваме, приемаме го и преминаваме към Teacher, след това Student.

Така накрая ще имаме реално **наша архитектура**, а не стандартен шаблон за роли.

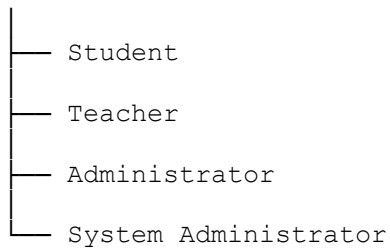
11. Вие казахте:

мисля че някой от ролите трябва да са към manager

12. ChatGPT каза:

Да — това е много добра идея. И тук според мен трябва да разграничим **Role / Роля** от **Manager / Мениджър**, защото не е задължително всяка роля да е директно в една и съща йерархия. Например можем да имаме:

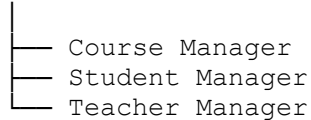
User



но някои от тях да имат **Manager / Управление** като допълнителна отговорност.

Например:

Manager



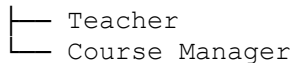
Това вече е различно от ролята Administrator.

Например

Един човек може да бъде:

User: Иван

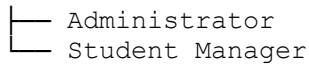
Roles:



А друг:

User: Мария

Roles:



Така не казваме:

„Всеки Administrator автоматично може всичко.“

Можем да разделим отговорностите.

Но преди да го направим така, бих те питал нещо важно:

Кои от трите роли според теб трябва да бъдат свързани с Manager / Мениджър?

Например имаш ли предвид:

- Course Manager / Мениджър на курсове
- Student Manager / Мениджър на студенти
- Teacher Manager / Мениджър на преподаватели

или имаш предвид **Manager като отделна роля**, която стои над някои от останалите?

13. Вие казахте:

да ги направим директно на 5 роли: системен админ, админ, manager, учител и студент

14.ChatGPT каза:

Да, това според мен е **по-чисто**, отколкото да се опитваме да натъпчем Manager като допълнителна функция към Administrator.

V1 — Roles / Роли

Фиксираме **5 основни роли**:

1. System Administrator / Системен администратор
2. Administrator / Администратор
3. Manager / Мениджър
4. Teacher / Преподавател
5. Student / Студент

Предварително разграничение

Роля	Основна отговорност
System Administrator	Техническо състояние и поддръжка на системата
Administrator	Административно управление на учебната система
Manager	Управление и контрол на курсове, хора и процеси
Teacher	Провеждане на обучение
Student	Участие в обучение

Но **още не фиксираме правата**.

Това е следващата стъпка — да определим **какво точно може всяка от петте роли**.

И тук според мен трябва да внимаваме особено с:

Administrator vs Manager

защото иначе двете роли лесно ще започнат да се припокриват.

Предлагам да започнем с тях двете и да определим много ясно **каква е разликата между Administrator и Manager**. След това Teacher и Student ще са доста по-лесни.

15.Вие казахте:

студента - създава и променя своя профил - име, фамилия и т.н. ;
има право да види към кои курсове иска да се запише, кой вече е записал и т.н.

16.ChatGPT каза:

Да. Това вече оформя доста ясно **Student / Студент** като роля. 👍

V1 — Student / Студент

1. My Profile / Моят профил

Студентът може да **създава и променя собствения си профил**.

Например:

First Name / Име
Last Name / Фамилия
Email
Phone
...

Тук правилото е:

Студентът може да променя **собствената си информация**, но не и профилите на други потребители.

2. Course Catalog / Каталог на курсовете

Студентът може да разглежда курсовете, които са достъпни за записване.

Например:

C# Основи

10 теми

Вечер

Hybrid

23 / 30 места

Може да вижда и информация като:

- име;
- описание;
- ниво;
- език;
- изисквания;
- начин на провеждане;
- дни;
- част от деня;
- свободни места;
- обща информация за курса.

3. Enrollment / Записване

Студентът може да:

- заяви записване за курс;
- избере Online / On-site при Hybrid, ако има такава възможност;
- види за кои курсове е записан;
- види статуса на записването си;
- отмени записването, **ако правилата на курса го позволяват.**

4. My Courses / Моите курсове

След записване студентът вижда:

My Courses

C# Основи

Status: In Progress

Progress: 7 / 10 — 70%

Attendance: 8 / 10 — 80%

Intermediate Grade: 5.25

Final Grade: —

Това е точно обобщената информация, която вече определихме за V1.

5. UI Settings / Настройки

Студентът има и собствените си UI настройки:

Theme

Language

Menu

Element Size

Layout

...

и може да превключва между:

Default

My Settings

Важно ограничение

Студентът **може да вижда информация за курсовете**, но не може да променя самия курс.

Тоест:

Student

— My Profile	→ Create / Edit
— Course Catalog	→ View
— Enrollment	→ Request / Manage own
— My Courses	→ View
— UI Settings	→ Edit own

А:

Course

Teacher

Other Students

System Settings

→ **няма право да ги редактира.**

Това бих го записал като основата на **Student / Студент — V1**.

След това можем да добавим само ако смяташ, че липсва нещо.

Student → **засега одобрено като концепция, но можем да го финализираме след като проверим дали искаш студентът да има още права.**

17. Вие казахте:

няма права да вижда профила на други студенти освен
определени статистики

18.ChatGPT каза:

Да, това е важно ограничение и го записваме изрично. 👍

V1 — Student / Студент — достъп

Студентът **няма право да вижда личния профил на друг студент.**

Тоест:

Student A

— My Profile	→ FULL ACCESS
— My Courses	→ FULL ACCESS
— Other Students	→ NO PROFILE ACCESS

Какво може да вижда за другите студенти?

Само **разрешената обобщена статистика**, без лични данни.

Например:

C# Основи — статистика

Students: 23
Completed: 8
In Progress: 15

Average Progress: 72%
Average Attendance: 84%

И евентуално:

Course Progress



Но **не**:

- име;
- фамилия;
- email;
- телефон;
- личен профил;
- индивидуални оценки;
- индивидуално присъствие;
- индивидуален прогрес.

Правилото става:

Student вижда собствените си лични и учебни данни, но за останалите студенти вижда само предварително разрешена агрегирана статистика.

Това е много добро и от гледна точка на **privacy / поверителност**.

OWN DATA

→ Full

OTHER STUDENTS

→ Aggregated Statistics Only

Това го считаме за част от **Student V1**. 

19. Вие казахте:

мисля че и с това приключихме ако нямаш ти да добавиш нещо

20. ChatGPT каза:

Да, според мен **Student / Студент за V1 е достатъчно добре изчистен**. Не бих добавял нови функции на този етап.

Само бих фиксираше едно важно архитектурно правило, което вече естествено излезе от разговора:

Student / Студент — V1

- Създава и редактира **собствения си профил**.
- Разглежда наличните курсове.
- Вижда към кои курсове може да се запише.
- Изпраща заявка/записва се за курс според правилата му.
- Вижда собствените си записвания и техния статус.
- Вижда собствените си курсове.
- Вижда:
 - прогрес;
 - оставащо;
 - присъствие;
 - междинна оценка;
 - крайна оценка.
- Може да избира собствените си UI настройки.
- **Не вижда профилите на други студенти.**
- За другите студенти вижда само **разрешена агрегирана статистика**.
- Няма право да променя данни за други студенти, курсове или преподаватели.

Единственото, което бих оставил за по-късно

Не бихме решавали още детайлно **как се одобрява записването**.

Например:

Student → заявка → Manager/Administrator → Approved

или:

Student → директно записване → ако има свободно място

Това според мен трябва да го решим, когато стигнем до **Manager / Мениджър**, защото там ще стане ясно кой контролира процеса.

Student V1 → приключен. 

Остават ни:

System Administrator	
Administrator	
Manager	
Teacher	
Student	

Следващият логичен е **Teacher / Преподавател** или **Manager / Мениджър**. Аз бих минал към **Teacher**, защото вече имаме доста ясна представа какво прави преподавателят с курса и студентите.

21. Вие казахте:

учител

22. ChatGPT каза:

Да, да минем на **Teacher / Преподавател**. Тук също предлагам да не бързаме с таблиците, а първо да определим **какво реално може и не може да прави ролята във V1**.

Teacher / Преподавател — V1

Аз бих започнал с тази основа:

1. My Profile / Моят профил

Преподавателят може да:

- създава своя профил;
- променя собствените си данни;
- променя собствените си UI настройки.

Teacher

└─ My Profile → Create / Edit own

2. My Courses / Моите курсове

Вижда курсовете/провежданията, към които е назначен.

Например:

C# Основи — 2026

Role: Teacher

Students: 18

Status: In Progress

Но тук трябва да решим нещо важно:

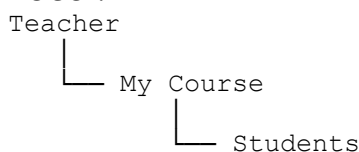
Може ли преподавателят сам да се добавя към курс, или това се прави от Administrator/Manager?

Аз бих предложил **не** — назначаването да се прави от Manager/Administrator.

3. Students / Студенти

Преподавателят може да вижда студентите **само в курсовете, които преподава.**

Тоест:



За тези студенти може да вижда необходимата учебна информация:

- име;
- прогрес;
- присъствие;
- междинна оценка;
- крайна оценка;
- статус.

Но не трябва автоматично да получава достъп до всички студенти в системата.

4. Attendance / Присъствие

Тук според мен преподавателят трябва да има право да **отбелязва присъствие** на студентите в своите курсове.

В V1:

Student: Иван
Attendance: 7 / 10

Преподавателят може да въвежда/коригира тази информация. Детайлното присъствие по конкретни занятия остава за V2.

5. Grades / Оценки

Преподавателят може да въвежда:

Intermediate Grade
Final Grade

за студентите от своите курсове.

Отново — във V1 са **обща стойности**, а не оценки от отделни изпити/теми.

6. Course information

Тук бих спрял и бихме го решили заедно.

Има голяма разлика между:

Teacher → View Course

и:

Teacher → Edit Course

Аз лично бих дал на преподавателя **ограничено редактиране**, но не бих му позволил да променя основните параметри като:

- име;
- капацитет;
- начин на провеждане;
- преподаватели;
- година.

Те по-скоро трябва да се управляват от Manager/Administrator.

Така първоначалният модел е

Teacher

— My Profile	→ Edit own
— UI Settings	→ Edit own
— My Courses	→ View
— My Students	→ View
— Attendance	→ Manage
— Grades	→ Manage

И не:

All Students

All Courses

System Settings

Other Teachers

Това е само първият вариант — **не го приемам като окончателен**. Нека сега го минем точка по точка, както направихме със Student, и ти кажеш какво би променил или добавил.

1. Вие казахте:

назначаването да се прави от Manager

2. ChatGPT каза:

Да. Това го фиксираме. 

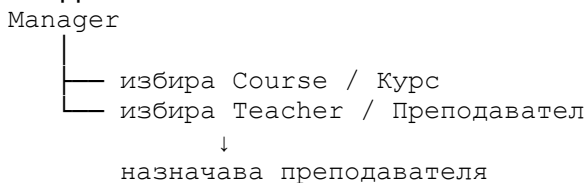
Teacher / Преподавател — назначаване

Teacher не може сам да се назначава към курс.

Назначаването се извършва от:

Manager / Мениджър

Моделът става:



Пример:

C# Основи — 2026

Teacher:

Иван Петров

Status: Assigned

И тъй като вече решихме, че един курс може да има повече от един преподавател:

C# Основи — 2026

Teachers:

Иван Петров

Мария Иванова

Manager → назначава и премахва преподаватели от конкретно провеждане.

Това е важно и за бъдещата V2 идея:

Teacher → конкретна тема/занятие

Там вече можем да определим кой от назначените преподаватели е водил конкретното занятие.

Teacher V1 — актуално

Teacher

My Profile	→ Edit own
UI Settings	→ Edit own
My Courses	→ View
My Students	→ View
Attendance	→ Manage
Grades	→ Manage

A:

Assign Teacher → Manager

Remove Teacher → Manager

Прието за V1. 

3. Вие казахте:

име на курс; капацитет; начин на провеждане; преподаватели; година това се определя от manager освен ако не е семинар

4. ChatGPT каза:

Да, това е важна особеност. Тогава трябва да направим разлика между **обикновен курс** и **семинар**.

Teacher / Преподавател — V1

За стандартен курс преподавателят **не определя** основните организационни параметри.

Те се управляват от **Manager / Мениджър**:

Course / Провеждане

— Име на курс	→ Manager
— Капацитет	→ Manager
— Начин на провеждане	→ Manager
— Преподаватели	→ Manager
— Година	→ Manager

Изключение — Seminar / Семинар

Тук преподавателят може да има по-голяма свобода.

Например:

Seminar

— Teacher	
— създава	
— определя име	
— определя капацитет	
— определя начин на провеждане	

Но тук възниква **много важен архитектурен въпрос**:

Какво точно разбираме под „семинар“?

Дали Seminar / Семинар е **вид курс**, който има различни правила?

Например:

Course Type

— Course / Курс
— Seminar / Семинар

Ако да, тогава правилата могат да бъдат:

Course

→ Manager controls organization

Seminar

→ Teacher can create/manage organization

Това според мен е по-чисто, отколкото да слагаме специални if seminar условия навсякъде в системата.

Но нека не го приемаме още. Първо да уточним какво точно искаш да може преподавателят да прави при семинар — само да създава неговите параметри, или и да го публикува/записва студенти и т.н.

5. Вие казахте:

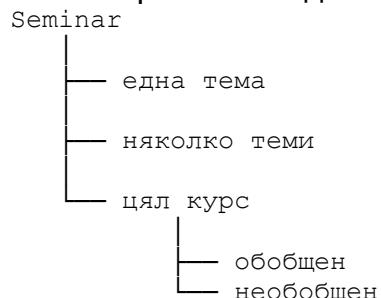
семинара е съвкупност от една или няколко тема или един курс - обобщен или не

6. ChatGPT каза:

А, така вече е много по-ясно. Значи **Seminar / Семинар** не е просто „вид курс“, а **самостоятелно събитие/обучение, което използва вече съществуващо учебно съдържание.**

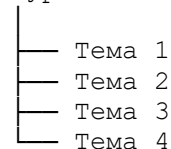
Seminar / Семинар — концепция

Семинарът може да съдържа:



Например:

Курс: C# Основи



Можем да направим:

Семинар А

→ Тема 2

→ Тема 3

или:

Семинар В

→ C# Основи — целият курс

А „обобщен“?

Тук разбирам идеята ти така:

Семинар

→ целият курс

→ представен като общо/обобщено съдържание

или:

Семинар

→ целият курс

→ без обобщаване

Това вече е много различно от Course / Курс.

И това обяснява защо преподавателят има изключение

При нормален курс:

Manager



определя организацията



Teacher



провежда курса

При семинар:

Teacher



създава Seminar



избира съдържанието



определя параметрите

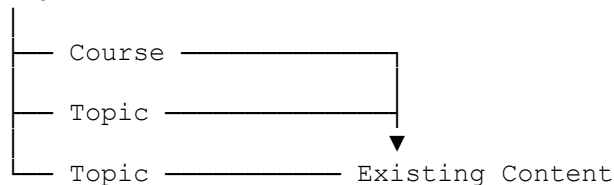
Това ми харесва като концепция.

Но бих предложил едно важно уточнение

Не бих копирал темите/курса в семинара.

По-добре семинарът да **реферира** към вече съществуващото съдържание:

Seminar



Така ако имаме:

C# Основи

и направим:

Семинар: Въведение в C#

не създаваме второ копие на съдържанието.

И тук според мен стигнахме до **нова важна концепция**, която досега не сме обсъждали:

Course / Курс и Seminar / Семинар вероятно трябва да бъдат две различни неща, но да могат да използват едно и също учебно съдържание.

Това е нещо, което бихме изчистили **преди да финализираме Teacher**, защото директно влияе на неговите права.

7. Вие казахте:

Семинарът е форма на групово обучение или научно занятие, при което малък брой участници обсъждат определена тема под ръководството на преподавател или експерт. Основни значения
Учебни занятия: Практически или дискуссионни групи в университет, където студентите участват активно.
Повишаване на квалификацията: Обучителни срещи за професионалисти за обмен на знания.
Помещение: Стая или зала, предназначена за такива групови събирания

8. ChatGPT каза:

Да — така вече имаме точната дефиниция и тя **коригира едно мое предишно предположение**: семинарът не трябва да се разглежда просто като „вид курс“.

Seminar / Семинар — V1

Приемаме:

Семинарът е форма на групово обучение или научно занятие, при което малък брой участници обсъждат определена тема под ръководството на преподавател или експерт.

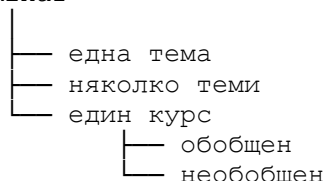
Може да се използва за:

- **Учебно занятие** — практическа или дискуссионна група;
- **Повишаване на квалификацията** — професионална обучителна среща;
- **Научно занятие** — обсъждане на определена тема;
- **Групова работа** под ръководството на преподавател/експерт.

Връзка с Course / Курс

Това, което каза по-рано, остава валидно:

Seminar



Но **семинарът е отделна форма на обучение**, а не просто разновидност на курса.

Например:

Course:

C# Основи

10 теми

↓

Seminar:

"Практическо приложение на C#"

съдържа:

– Тема 4

– Тема 7

– Тема 9

или:

Seminar:

"Обобщение на C# Основи"

Content:

→ целият курс

→ обобщен

И тук идва важната връзка с Teacher

При **Course**:

Manager



определя организацията



Teacher



провежда

При **Seminar** може да имаме:

Teacher / Expert



създава Seminar



избира съдържанието



определя параметрите



провежда Seminar

Това обяснява напълно защо искаш **изключението „ако не е семинар“**.

Само едно нещо бих предложил да не решаваме още

Не бих фиксирал засега дали семинарът може да бъде създаден само от Teacher, или и от Manager.

Това ще стане по-ясно, когато разгледаме **Manager / Мениджър**.

Засега можем да приемем:

Seminar / Семинар → отделна форма на обучение, базирана върху една/няколко теми или курс, с малка група участници и преподавател/експерт. 

И това е важна част от **Teacher V1**, защото вече имаме реална причина преподавателят да има различни права при Course и Seminar.

1. manager може ли според теб

2. **ChatGPT каза:**

Да — според мен можем да го направим. И точно **Manager / Мениджър** е ролята, която трябва да свърже организационната част между Administrator, Teacher и Student.

Аз бих предложил първо да определим **какво управлява Manager**, а после да решим какво остава изцяло за Administrator.

Manager / Мениджър — V1

Основна идея

Manager управлява конкретните курсове и процеса около тях.

Тоест не управлява технически системата и не е задължително да има всички права на Administrator.

За Course / Курс

Manager може да:

- създава курс;
- редактира информацията за курса;
- определя:
 - име;
 - описание;
 - изисквания;
 - ниво;
 - език;
 - общ брой теми;
 - година;
 - капацитет;
 - начин на провеждане;
 - дни;
 - сутрин/следобед/вечер;
- назначава преподаватели;
- премахва преподаватели;
- управлява записването на студенти.

За Students / Студенти

Manager може да:

- вижда студентите в управляваните от него курсове;
- одобрява/отхвърля заявки за записване;
- записва студент;
- премахва студент от курс;
- вижда общия напредък;
- вижда присъствието;
- вижда оценките.

Но тук бих предложил **да не му даваме право да променя оценки и присъствие**, защото това е работа на Teacher.

За Teachers / Преподаватели

Manager:

- избира преподавател;
- назначава го към курс;
- премахва го от курс;
- вижда преподавателите;
- вижда към кои курсове са назначени.

Какво не прави Manager

System Administration → System Administrator

Teaching → Teacher

Technical configuration → System Administrator

И тук идва най-интересната част:

Administrator / Администратор.

Защото трябва да решим дали Administrator е:

Administrator

↓

има всички права на Manager

+

допълнителни административни права

или двете роли са **паралелни**, с различни отговорности.

Аз лично бих предпочел първия вариант — **Manager управлява оперативно курсовете, а Administrator има по-високо ниво на управление**, но нека го обсъдим, преди да го приемаме.

3. Вие казахте:

за мен админ-ще определя права, маха права и от там ще се изкл/вкл отделните линкове към формите или ще ги скрива за по голяма сигурност както кой е учител и кой е manager, ако това е необходимо

4. ChatGPT каза:

Да — това според мен прави ролята **Administrator / Администратор** много по-ясна. И има важна разлика между Administrator и Manager.

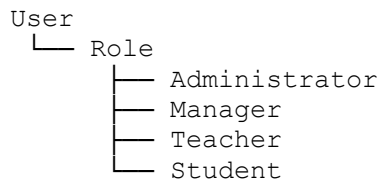
Administrator / Администратор — V1

Основната му задача е:

Управление на достъпа и правата в системата.

1. Roles / Роли

Администраторът може да определя ролите на потребителите:



Например:

Иван
Role: Teacher

може да бъде променен на:

Иван
Role: Manager

или при необходимост:

Иван
Roles:

- Teacher
- Manager

Това последното бихме го запазили като възможност, ако решим, че потребител може да има повече от една роля.

2. Permissions / Права

Администраторът определя **какво има право да прави дадена роля**.

Например:

Teacher

View Courses	✓
Edit Course	X
Manage Attendance	✓
Manage Grades	✓

Докато:

Manager

View Courses	✓
Edit Course	✓
Assign Teacher	✓
Manage Students	✓

3. UI / Достъп до формите

Тук идеята ти е особено полезна.

Ако дадена функция не е разрешена:

Permission: false

системата може:

Dashboard

Courses	✓
Students	✓
Teachers	X
Administration	X

и съответните линкове/бутони **да не се показват**.

Това прави интерфейса по-чист и намалява вероятността потребителят да стига до нещо, което не му е позволено.

Но има едно много важно правило

Скриването на линка НЕ е самата сигурност.

Например:

Teacher

не вижда:

/Administration

но ако директно въведе адреса, сървърът пак трябва да провери:

User

↓

Role

↓

Permission

↓

Allow / Deny

Тоест:

UI скрива → Security layer забранява.

Това е много важно архитектурно решение.

Manager vs Administrator

Така вече разликата става доста добра:

System Administrator

→ техническо състояние на системата

Administrator

→ права, роли и достъп

Manager

→ управление на курсове, преподаватели и студенти

Teacher

→ обучение, присъствие и оценки

Student

→ собствено обучение

И бих добавил още едно правило към Administrator:

Administrator управлява правата, но не трябва автоматично да получава всички технически права на System Administrator.

Така разделението на отговорностите остава чисто.

Това според мен вече е **много добра основа за Administrator**

V1. Следващото, което трябва да уточним, е **дали правата се задават директно на роли или Administrator може да дава и индивидуални права на конкретен потребител.** Това ще е важно за архитектурата.

1. може да го оставим за v2

2. ChatGPT каза:

Да, точно така. 👍 Това е добър кандидат за V2, защото V1 ще остане по-проста и предвидима.

Administrator / Администратор — V1

Фиксираме:

- управлява **ролите** на потребителите;
- определя **правата на ролите**;
- може да включва/изключва достъпа до определени функции/форми;
- според правата системата може да **показва или скрива съответните менюта и линкове**;
- реалната проверка на правата се извършва от системата, а не само чрез скриване на UI;
- не получава автоматично техническите права на System Administrator.

V2

Оставяме:

- индивидуални права за конкретен потребител;
- изключения от правата на ролята;
- по-сложна система за наследяване на права;
- евентуално временни права.

Така V1 остава:

```
User
↓
Role
↓
Permissions
↓
Access
```

а V2 може да стане:

```
User
↓
Role
↓
Role Permissions
+
User-specific Permissions
↓
Final Access
```

Administrator V1 → приет. 

Вече имаме изчистени **4 от 5 роли**:

System Administrator	✓
Administrator	✓
Manager	⌚
Teacher	✓
Student	✓

Остава ни само **Manager / Мениджър**, след което можем да направим цялостна проверка на петте роли и да видим дали някъде има припокриване или липсващо право.

3. Вие казахте:

за manager оставам както ти предложи

4. ChatGPT каза:

Да, тогава го фиксираме така. 👍

Manager / Мениджър — V1 ✓

Основната му задача е **оперативното управление на курсовете и участниците в тях.**

Course / Курс

Manager може да:

- създава курс;
- редактира курса;
- определя:
 - име;
 - описание;
 - изисквания;
 - ниво;
 - език;
 - общ брой теми;
 - година;
 - капацитет;
 - начин на провеждане;
 - дни;
 - сутрин / следобед / вечер.

Teachers / Преподаватели

Manager:

- избира преподаватели;

- назначава преподаватели към курс;
- премахва преподаватели от курс;
- вижда кои преподаватели са назначени към даден курс.

Students / Студенти

Manager:

- вижда студентите в управляваните курсове;
- одобрява/отхвърля заявки за записване;
- записва студент;
- премахва студент от курс;
- вижда общия прогрес;
- вижда присъствието;
- вижда оценките.

Но:

Manager не променя оценките и присъствието — това остава отговорност на Teacher.

Какво не може Manager

System Status / Maintenance
→ System Administrator

Roles / Permissions
→ Administrator

Teaching / Attendance / Grades
→ Teacher

Така получаваме много чисто разделение:

System Administrator
→ Системата

Administrator
→ Достъпът

Manager
→ Организацията

Teacher
→ Обучението

Student
→ Участието

V1 — Петте роли вече са изчистени 

1. System Administrator
2. Administrator
3. Manager

4. Teacher

5. Student

Според мен **не трябва да добавяме повече права към Manager засега**. По-добре следващата ни стъпка да бъде да направим **пълна матрица „Role x Permission“ за V1**, за да видим на едно място дали има конфликт или дупка в архитектурата.

5. Вие казахте:

ОК

6. ChatGPT каза:

Супер. 👍 Тогава следващата точка е **матрицата на ролите и правата за V1**.

Преди да я направим окончателна, бих я подредил по функционалности, а не по таблици. Така много по-лесно ще видим припокриванията:

Функционалност	System Admin	Admin	Manager	Teacher	Student
System Status	✓	👁	👁	👁	👁
Maintenance	✓	✗	✗	✗	✗
System Announcement	✓	✗	✗	✗	✗
Управление на роли	✗	✓	✗	✗	✗
Управление на права	✗	✓	✗	✗	✗
Създаване на курс	✗	според правата	✓	✗	✗
Редактиране на курс	✗	според правата	✓	✗	✗
Назначаване на преподавател	✗	според правата	✓	✗	✗
Записване на студент	✗	според правата	✓	✗	собствено
Присъствие	✗	според правата	👁	✓	собствено
Оценки	✗	според правата	👁	✓	собствени
Собствен профил	✗	собствен	собствен	собствен	собствен
UI настройки	✗	собствен	собствен	собствен	собствен
Курсове	системен статус	управление според права	управление	свои налични/свои	
Статистика	✗ според права		свои курсове	свои курсове	разрешена

Тук има една важна точка за обсъждане: Administrator не трябва да бъде автоматично „може всичко“. Вече решихме, че той **определя правата**, затова в матрицата неговият достъп до конкретните функции може да зависи от зададените му permissions.

A Manager, Teacher и Student имат основните си фиксирани функции. Следващата стъпка бих я направил **точно тази таблица да я минем ред по ред**, за да не остане някое право неуточнено.

7. Вие казахте:

може ли да направиш като таблица функционалност, другите колони всички роли

8. ChatGPT каза:

Да. Така ще е много по-удобно за преглед. Ще използвам:

- ✓ — има право
- 👁 — само вижда
- — — няма достъп
- **собствен** — само собствените данни
- **според права** — зависи от Permission, зададен от Administrator

V1 — Role × Functionality

Функционалност	System Administrator	Administrator	Manager	Teacher	Student
System Status / Състояние на системата	✓	👁	👁	👁	👁
Maintenance Mode / Поддръжка	✓	—	—	—	—
Maintenance Message / Съобщение при поддръжка	✓	—	—	—	—
System Announcement / Системно съобщение	✓	—	—	—	—
Roles / Управление на роли	—	✓	—	—	—

Функционалност	System Administrator	Administrator	Manager	Teacher	Student
Permissions / Управление на права	—	✓	—	—	—
User Profile / Собствен профил	собствен	собствен	собствен	собствен	Собствен
UI Settings / UI настройки	собствен	собствен	собствен	собствен	Собствен
Create Course / Създаване на курс	—	според права	✓	—	—
Edit Course / Редактиране на курс	—	според права	✓	—	—
View Course / Преглед на курс	—	според права	✓	свои	✓
Course Capacity / Капацитет	—	според права	✓	👁	👁
Course Type / Начин на провеждане	—	според права	✓	👁	👁
Course Year / Година	—	според права	✓	👁	👁
Assign Teacher / Назначаване на преподавател	—	според права	✓	—	—
Remove Teacher / Премахване на преподавател	—	според права	✓	—	—
View Students / Преглед на студенти	—	според права	✓	свои курсове	Статистика
Enroll Student / Записване на студент	—	според права	✓	—	Собствено
Approve Enrollment / Одобряване на записване	—	според права	✓	—	—
Remove Student / Премахване от курс	—	според права	✓	—	—
Attendance / Присъствие	—	според права	👁	✓	собствено
Grades / Оценки	—	според права	👁	✓	собствени

Функционалност	System Administrator	Administrator	Manager	Teacher	Student
Progress / Прогрес	—	според права	👁	✓	собствен
Course Statistics / Статистика за курс	—	според права	✓	✓	разрешена
Other Student Profiles / Профили на други студенти	—	според права	✓	свои курсове	—
Other Student Statistics / Статистика за други студенти	—	според права	✓	свои курсове	агрегирана
System Administration / Системна администрация	✓	—	—	—	—

Едно важно уточнение

При Administrator „според права“ не означава автоматично **достъп**. Означава, че Administrator може да конфигурира дали съответното право е разрешено.

Например:

Permission: Edit Course

Administrator → ✓

Manager → ✓

Teacher → —

Student → —

И ако по-късно Administrator промени permission-a:

Manager → Edit Course → —

системата трябва едновременно:

- да скрие съответната функционалност от UI;
- да блокира директния достъп до нея.

Така таблицата може да стане **основната V1 матрица**, от която после да проектираме permissions и достъпа до формите.

1. това е добре за момента като в всеки момент може да се промени като я оставяме проста, изчистена и гъвкава

2. ChatGPT каза:

Точно така. Това според мен трябва да бъде един от основните принципи на проекта:

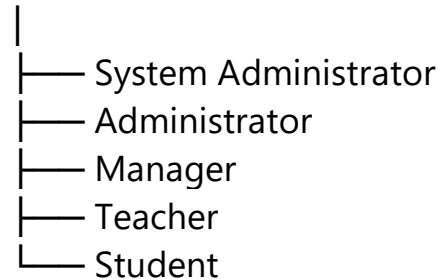
V1 да бъде проста, изчистена и гъвкава, без да заключаваме архитектурата за бъдещите версии.

V1 — Role & Permission принцип

За момента имаме базова матрица, но **не я приемаме като нещо необратимо.**

V1

Roles



Simple Permissions



Access to functionality

Ако в процеса на разработката установим, че:

- дадено право липсва;
- дадено право е излишно;
- две права трябва да се обединят;
- някоя роля трябва да получи/загуби достъп;
- дадена функционалност всъщност е за V2;

просто променяме модела.

Основното правило

Не проектираме V1 така, сякаш знаем предварително как ще изглежда V3.

Правим достатъчно добра основа, върху която после можем да надграждаме.

Това пасва много добре и на решенията ни досега:

- **V1** → Course / Курс;
- **V2** → повече детайли + Seminar / Семинар;
- **V3** → евентуално независими модули;
- ролите остават прости;
- permissions могат да се разширяват;
- UI настройките са отделени;
- бъдещите функции не се натъпкват преждевременно във V1.

Това бих го записал като архитектурен принцип на проекта:

Simple → Clean → Flexible → Extensible

Просто → Изчистено → Гъвкаво → Разширяемо

И според мен можем да продължим към следващата точка от архитектурата.

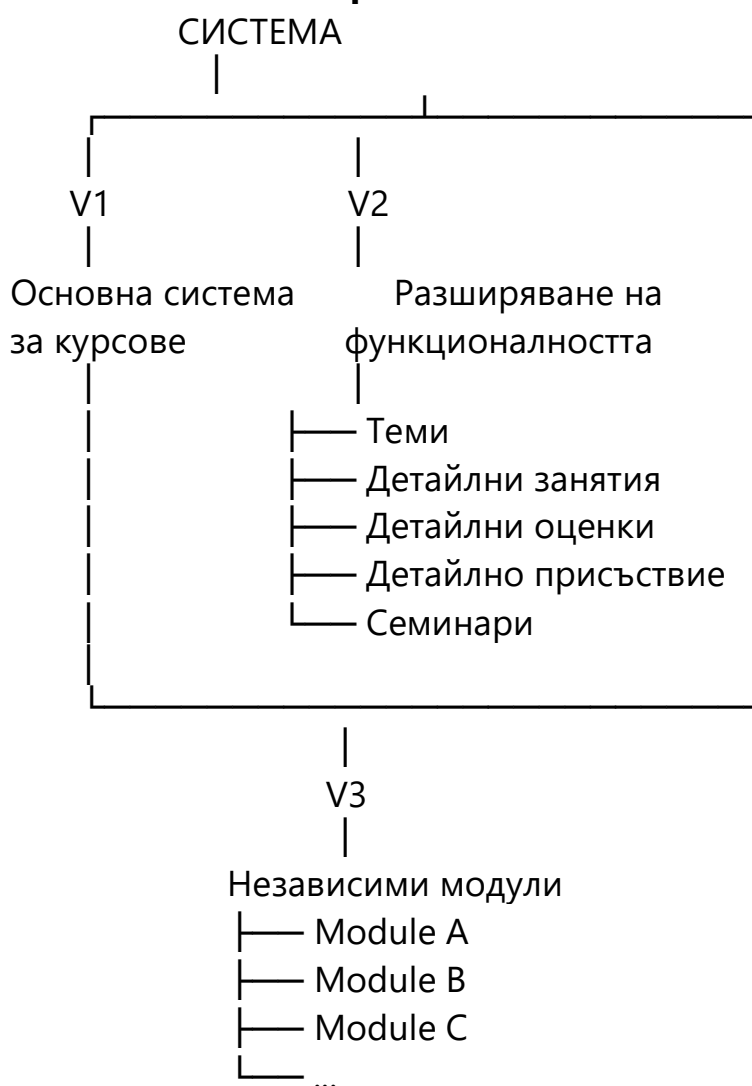
3. Вие казахте:

може ли да направиш обобщение на плана, архитектира и таблици, блок схеми

4. ChatGPT каза:

Да. Вече имаме достатъчно решения, за да направим **обща V1 картина**. Ще я разделя на: план, архитектура, роли, таблици и блок-схеми. Там, където още не сме взели окончателно решение, ще го отбележа като **за обсъждане**, вместо да го измислям.

1. Основен план на проекта

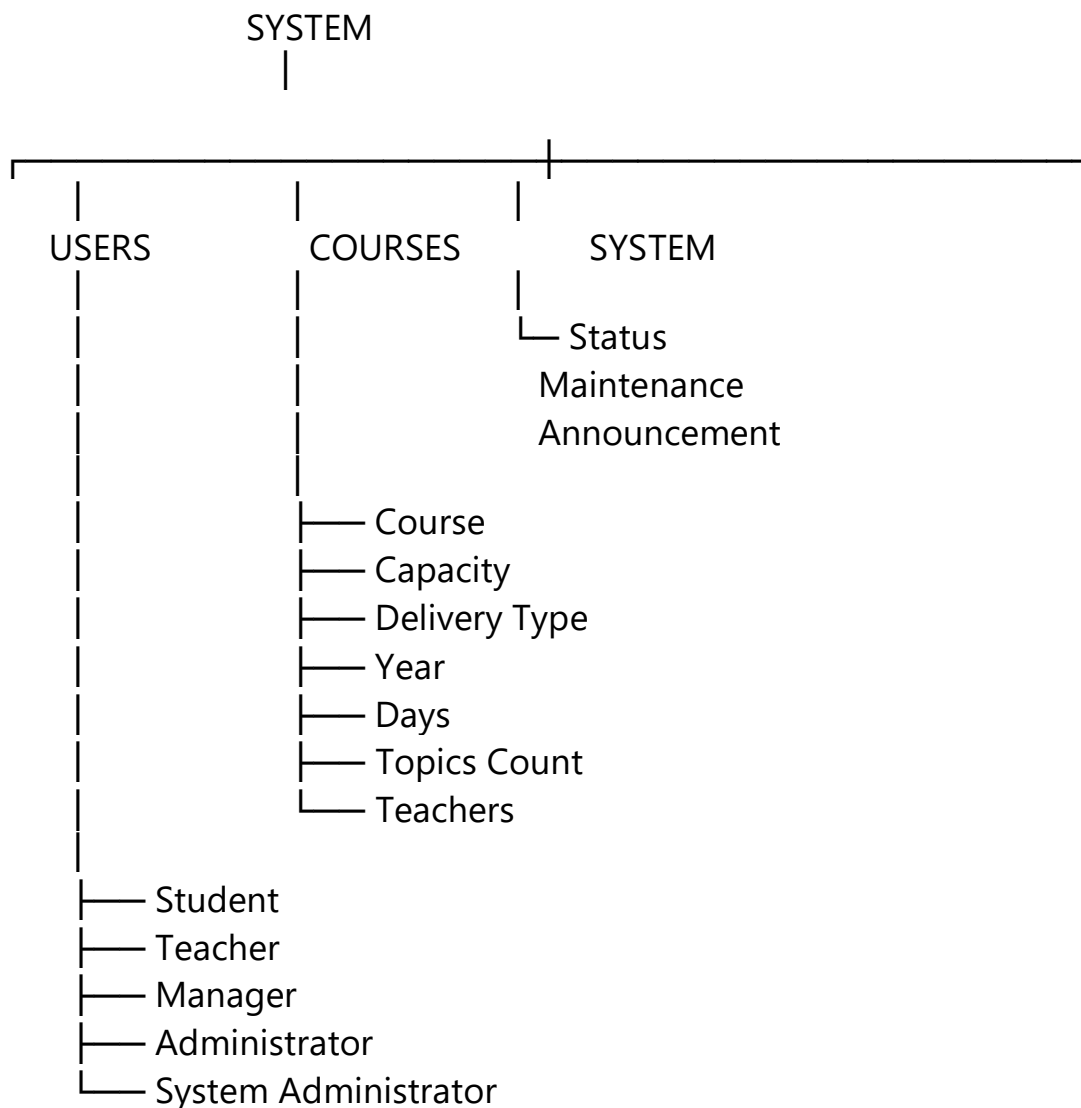


Основният принцип:

V1 → проста, изчистена и гъвкава основа.

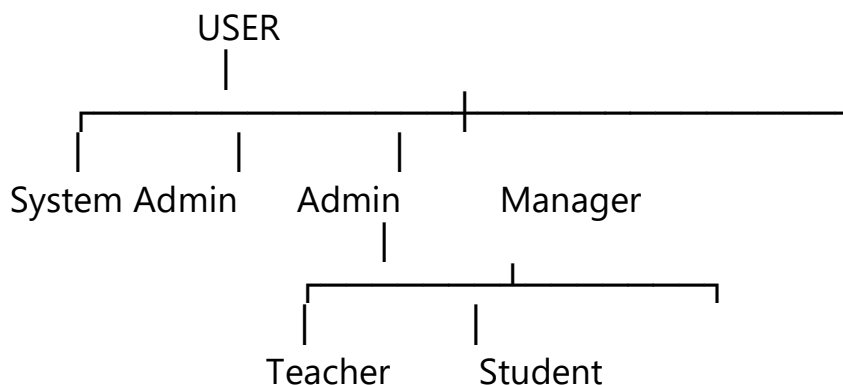
Не изграждаме предварително всичко за V2/V3.

2. V1 — Основна архитектура



3. Роли — V1

Имаме **5 роли**:



Функционално ги разграничихме така:

Роля

**System
Administrator**

Основна отговорност

Състояние и поддръжка на
системата

Роля
Administrator

Manager

Teacher

Student

Основна отговорност

Роли, права и достъп

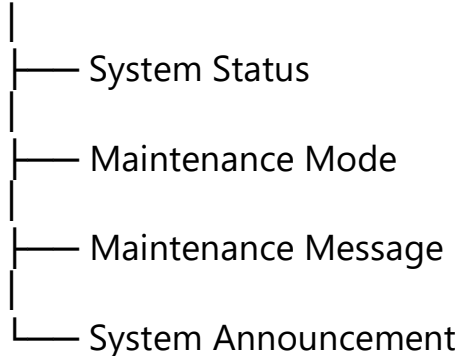
Управление на курсове и
участници

Обучение, присъствие и оценки

Участие и собствено обучение

4. System Administrator

System Administrator



V1

Само базова системна администрация.

V2/V3

Оставяме за по-късно:

Logs

Monitoring

Advanced Configuration

Module Management

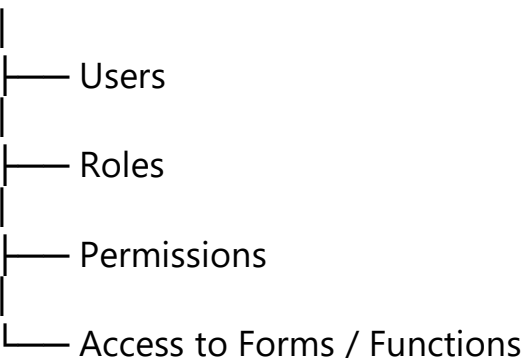
Independent Updates

Fault Isolation

5. Administrator

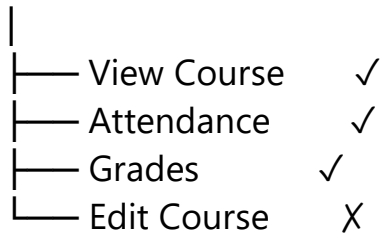
Основната му функция е **управление на достъпа**.

Administrator



Например:

Teacher



UI:

Permission = TRUE



показва се функцията

Permission = FALSE



скрива се функцията



сървърът блокира достъпа

Важно: скриването на линк не е механизъм за сигурност. Правата трябва да се проверяват и на ниво backend.

V2

Оставяме:

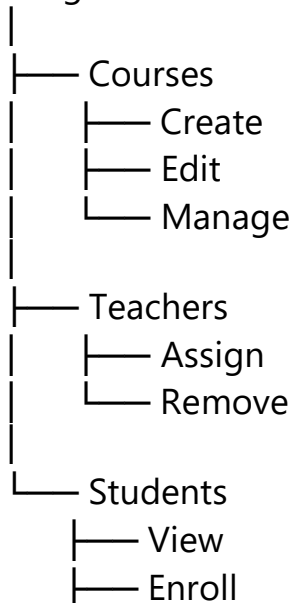
Role Permissions



Individual User Permissions

6. Manager

Manager



- └─ Approve
- └─ Reject
- └─ Remove

Manager определя основните параметри на курса:

Course

- └─ Name
- └─ Description
- └─ Requirements
- └─ Level
- └─ Language
- └─ Topics Count
- └─ Year
- └─ Capacity
- └─ Delivery Type
- └─ Days / Time period

Teacher **не се назначава сам.**

Manager

- └─ Assign Teacher

↓

Course

Manager вижда учебната информация, но **не въвежда оценките и присъствието.**

7. Teacher

Teacher

- └─ My Profile
- └─ UI Settings
- └─ My Courses
- └─ My Students
- └─ Attendance
- └─ Grades

Teacher работи само с курсовете, към които е назначен.

V1

Присъствие:

7 / 10

70%

Оценки:

Intermediate Grade

Final Grade

V2

Ще можем да преминем към:

Course



Topic



Lesson



Teacher

и да определяме **кой преподавател е водил конкретна тема/заяние**.

8. Student

Student



├── My Profile



├── Course Catalog



├── Enrollment



├── My Courses



├── Progress



├── Attendance



├── Grades



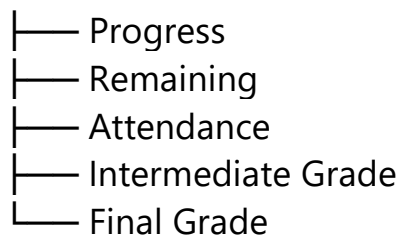
└── UI Settings

Студентът вижда:

Course



├── Status



Например:

Progress:

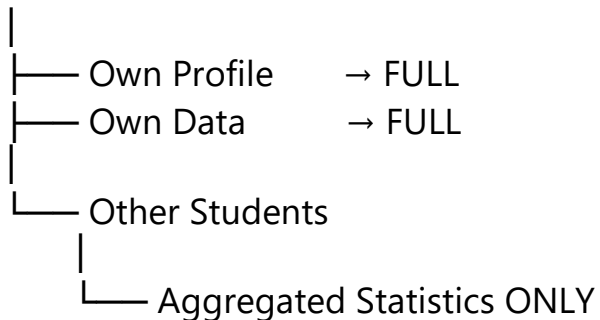
7 / 10 — 70%

Attendance:

8 / 10 — 80%

Поверителност

Student A

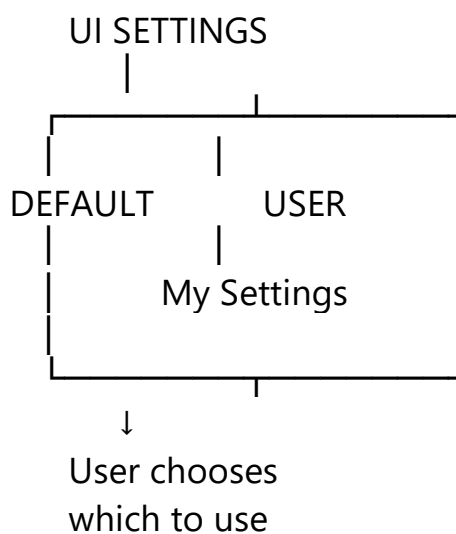


Не вижда личните профили, оценки, присъствие или индивидуален прогрес на други студенти.

9. UI Settings

Това е отделна концепция, която важи за потребителите.

Имаме:



Default

Системно зададени настройки.

User

Личните настройки на потребителя:

Theme

Language

Menu

Element Size

Layout

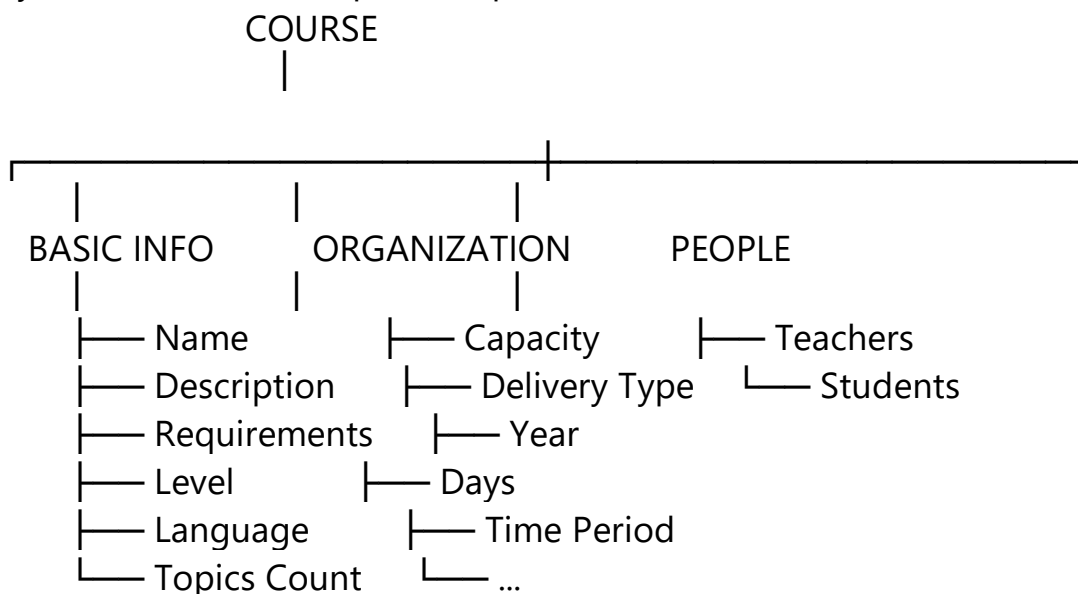
Other Visual Preferences

И двете конфигурации **се запазват**.

Потребителят може да превключва между тях.

10. Course — V1

Тук е същината на първата версия.



Важно:

V1 има само обща информация за темите.

Например:

Topics Count = 10

а не:

Topic 1

Topic 2

Topic 3

...

Детайлите са за V2.

11. Student ↔ Course

Тук имаме свързващата логика:

Student



Enrollment



Course

Един студент може да има:

Course A → Completed

Course B → In Progress

Course C → Enrolled

A за текущ курс:

Progress:

7 / 10 — 70%

Attendance:

8 / 10 — 80%

Intermediate:

5.25

Final:

—

Това е важна част от модела.

12. Course ↔ Teacher

Course



Teacher(s)

Назначаването:

Manager



Assign Teacher



Course

He:

Teacher



Assign Self

13. Основни таблици — V1

На база на всичко, което сме обсъдили, бих започнал с приблизително следната структура:

USER

- └─ Role
- └─ User Settings

COURSE

- └─ Course information
- └─ Delivery Type
- └─ Course status/information

TEACHER

- └─ User relationship

STUDENT

- └─ User relationship

COURSE_TEACHER

- └─ Course ↔ Teacher

COURSE_STUDENT

- └─ Course ↔ Student

ATTENDANCE

- └─ Student ↔ Course

GRADES

- └─ Student ↔ Course

Тук **не бих фиксирам окончателно имената и броя на таблиците още**, защото предстои да разгледаме реалния V4.txt и текущата структура на проекта.

14. Какво НЕ слагаме във V1

Това е също толкова важно.

V1

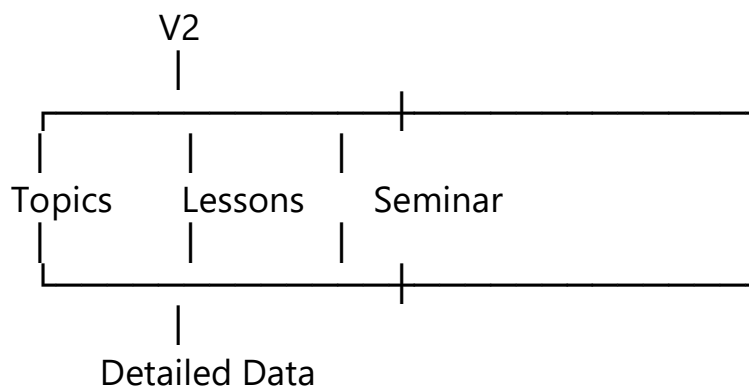
- └─ ✗ Detailed Topics
- └─ ✗ Detailed Lessons
- └─ ✗ Teacher per Topic

- ✗ Seminar
- ✗ Individual Permissions
- ✗ Independent Modules
- ✗ Module Fault Isolation
- ✗ Advanced Monitoring
- ✗ Complex Scheduling

Те не са забравени — **умишлено са оставени за бъдещи версии.**

15. V2

След като V1 работи, можем да развием:



Тогава числата от V1 могат да станат **изчисляеми**.

Например вместо:

7 / 10

70%

системата реално да изчислява:

Completed Topics

/

Total Topics

↓

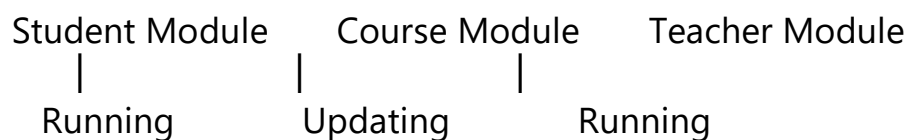
Percentage

Същото за присъствие и оценки.

16. V3 — евентуално

Тук е идеята, която обсъдихме, но **не я заключваме като задължителна**:





Ако:

Course Module



Problem

останалите:

Student Module → Running

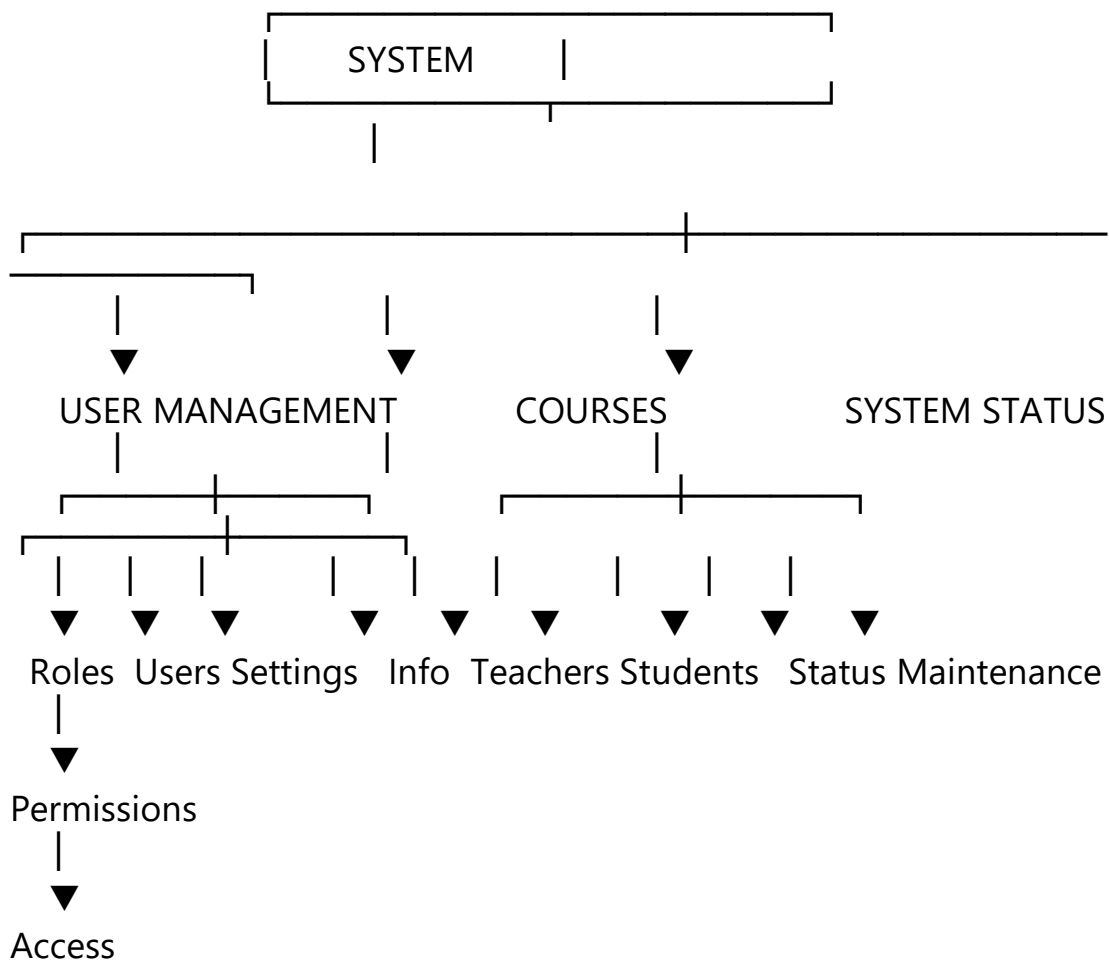
Teacher Module → Running

продължават да работят.

Това вече е бъдеща архитектурна стъпка, която ще решим след натрупания опит от V1/V2.

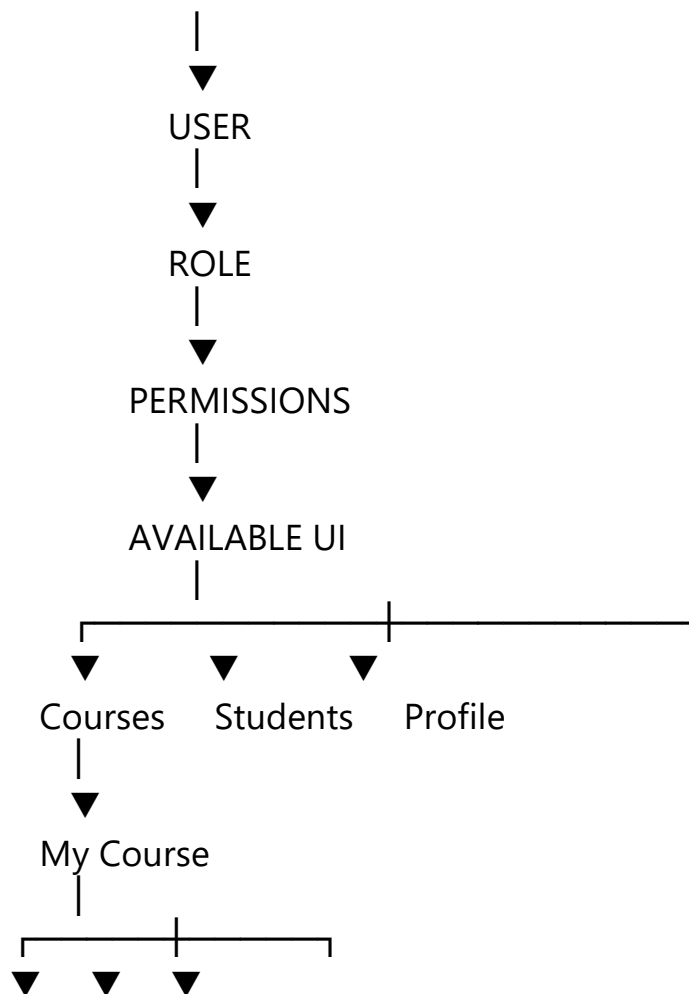
17. Общата блок-схема

Най-кратко цялата система в момента изглежда така:



И потребителският поток:

LOGIN



Progress Attendance Grades

Финално състояние на архитектурата

В момента имаме доста добра основа:

V1 = Course-centered system / Система, центрирана около курсовете

с:

- 5 роли;
- ясни основни права;
- Course ↔ Student;
- Course ↔ Teacher;
- прогрес;
- присъствие;
- оценки;
- записване;
- UI настройки;
- системен статус;
- проста permission система;
- ясна граница V1/V2/V3.

И най-важното решение, което според мен трябва да запазим като **водещ принцип**:

Не добавяме функционалност само защото технически можем. Добавяме я, когато реално е необходима.

Това ще ни помогне V1 да остане **малка, разбираема, стабилна и лесна за разширяване.**